KIDNEY DISEASE CLASSIFICATION USING CT SCAN IMAGES

Project report submitted
For Review (Major Project) in fulfilment of
The requirement for the degree of

Bachelor of Technology

(Computer Science and Engineering)

By
Saumyajeetsinh Vaghela - 21124061
Om Mallick - 21124070
Hiya Shah - 21124073
Pujan Gandhi - 21124093

Guided By: Dr. Swati Rai

Department of Computer Science and Engineering
School of Engineering and Technology
Navrachana University, Vadodara
May 2025

CERTIFICATE

This is to certify that the project report entitled “KIDNEY DISEASE CLASSIFICATION USING CT SCAN IMAGES” submitted by “Saumyajeetsinh Vaghela, Om Mallick, Hiya Shah and Pujan Gandhi” to School of Engineering and Technology (SET) of Navrachana University Vadodara, in partial fulfillment for the award of the degree of B. Tech in Computer Science and Engineering (CSE) department. This report is a bona fide record work that has been carried out under my supervision during academic year 2024-25. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree or diploma.

Prof. Swati Rai
Project Guide and Assistant Professor
Computer Science and Engineering
School of Engineering and Technology
Navrachana University, Vadodara

(_____)
Industry Expert
External Evaluator
Designation: _____
Company: _____

Prof. Ninad Bhavsar
Program Chair and Assistant Professor
Computer Science and Engineering
School of Engineering and Technology
Navrachana University, Vadodara

Dr. Ashish Jani
Head and Professor
CSE, BCA and BSc DS
School of Engineering and Technology
Navrachana University, Vadodara

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the sources. We also declare that we have adhered to all academic honesty and integrity principles and have not misrepresented, fabricated, or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Name of the student	**Student ID**	**Signature**
Saumyajeetsinh Vaghela	21124061	
Om Mallick	21124070	
Hiya Shah	21124073	
Pujan Gandhi	21124093	

Date: _____

ACKNOWLEDGEMENT

Every project big or small is successful largely due to the effort of several wonderful people who have always given their valuable advice or lent a helping hand. We sincerely appreciate the inspiration, support and guidance of all those people who have been instrumental in making this project a success. We are extremely grateful to **Prof. Ashish Jani**, (Head- CSE, BSc-DS, BCA) and **Prof. Ninad Bhavsar (**Program Chair- CSE) for entrusting our project "KIDNEY DISEASE CLASSIFICATION USING CT SCAN IMAGES" and for allowing us to undertake such kind of challenging and innovative work. We feel deeply honoured to express our sincere thanks to our Internal Project Guide **Prof. Swati Rai** for providing valuable insights leading to the successful completion of the project.

We would also like to thank all the CSE faculty members for their critical advice and guidance, without which this project would not have been possible.

Last but not least, we are deeply grateful to colleagues and friends who have been a constant source of inspiration during the preparation of this project.

Saumyajeetsinh Vaghela - 21124061
Om Mallick - 21124070
Hiya Shah - 21124073
Pujan Gandhi - 21124093

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

Manually interpreting medical imaging to classify and diagnose kidney diseases has traditionally been time consuming and prone to human error. In this project, we try to solve the challenges by building a deep learning based solution for automated classification of kidney diseases from CT scan images. We concentrate on building CNN models that can accurately detect and classify several types of kidney disease in order to ultimately improve diagnostic accuracy and reduce the work load of healthcare providers.

In line with modern MLOps (Machine Learning Operations) practices, the methodology pairs a robust machine learning pipeline with tools aimed at making model development, deployment and scaling a non event. For efficient experiment tracking and clear comparison between model versions, the project leverages MLflow, and for managing and versioning large datasets between various stages of the model development lifecycle, we use DVC (Data Version Control), to guarantee data integrity. This is important in clinical settings where reliability of models is essential, and reproducibility is paramount.

In addition, the deployment architecture has scalability and accessibility in mind, and utilizes the AWS cloud infrastructure. The model and its dependencies will be encapsulated in Docker containers which makes provisioning and distribution of the solution to any given healthcare facility an easy affair. Containers are also helpful to scale the system up, responding to increased demand with no sacrifice in performance. By fitting the system into such healthcare providers' existing infrastructure with minimal friction, services to other than general research and development will not just persist, but can also be proliferated in real clinical environments.

The goal of this project is to improve the efficiency and effectiveness of kidney disease management by combining state of the art deep learning techniques with deployment practices that are practical.

Keywords: Deep Learning, Medical Image Analysis, Kidney Disease, CNN, MLOps, MLflow, DVC, Docker, AWS, Healthcare Automation

TABLE OF CONTENTS

1. Kidney Disease Classification Using CT Scan Images

The key contributions of this project are focused towards development and deployment of a system for categorising kidney diseases based on CT scan images using state of the art deep learning techniques. This work spans the entire end to end process including data acquisition and preprocessing, model development, model evaluation, and deployment in a real world healthcare setting.

The system is designed to enable medical professionals to automatically interpret medical images so that kidney disease can be diagnosed faster and more accurately. The project brings together several machine learning and operations tools to make sure the model is scalable, reproducible and easy to deploy. The system specifically uses a Convolutional Neural Network (CNN) to classify images at high precision in order to identify abnormalities in kidney CT scans.

Finally, the implementation uses MLflow for experiment tracking and DVC (Data Version Control) for managing datasets are incorporated. With these tools, the project guarantees transparent development which tracks models, parameters, and data versions consistently. It also further increases the efficiency in which you can iterate and improve the model.

Deployment is another critical part of scope since the system will be containerized using the Docker and deployed on AWS (Amazon Web Services). This makes scalability as well as access in a variety of healthcare settings from small clinics to large hospitals possible, and avoids the system being overwhelmed by varying demand. The goal is then to make the system practical for use in real world settings, thereby reducing the diagnostic burden on radiologists and improving patient outcomes by increasing the speed and reliability of diagnosis.

Project Definition:

The project can be defined as a comprehensive solution for automating the classification of kidney diseases through deep learning. It involves the construction of a CNN model trained on CT scan images, which can recognize and categorise different kidney diseases. The project is divided into multiple stages:

1. **Data Acquisition and Preprocessing**: Gathering and preparing a large dataset of kidney CT scans for training and evaluation.
2. **Model Development**: Designing and implementing a CNN model capable of identifying patterns in medical images indicative of kidney diseases.
3. **MLOps Integration**: Utilising tools like MLflow and DVC for efficient management of experiments and data to ensure reproducibility.
4. **Deployment**: Using Docker containers to package the system for deployment on AWS, enabling cloud-based access and scaling.
5. **Evaluation**: Testing the model's accuracy, precision, and recall of unseen data to ensure its effectiveness in real-world scenarios.

By focusing on these key stages, the project delivers a robust, scalable solution for automating a critical component of kidney disease diagnosis. This system can be integrated into existing healthcare workflows, helping radiologists make quicker and more informed decisions.

Project Scope

The scope of this project includes:

1. **Medical Use Case Automation**
   Automating the diagnosis of kidney conditions using CT scan images, reducing manual diagnostic time and workload for radiologists.

2. **Model Design and Development**
   Designing a scalable and interpretable deep learning model using CNNs, tailored to classify medical images accurately.

3. **Experiment Management**
   Applying MLOps principles to ensure traceability of model versions, experiments, and data lineage through MLflow and DVC.

4. **Robust Dataset Handling**
   Building a reproducible pipeline for managing large image datasets, including preprocessing, augmentation, and versioning.

5. **Cloud-Ready Deployment**
   Deploying the model on a scalable cloud environment (AWS), containerized via Docker, with support for API-based access.

6. **Clinical Integration Readiness**
   Ensuring the system can be integrated into clinical workflows by using a real-time API, delivering predictions instantly to assist medical decisions.

7. **Performance Evaluation**
   Testing the model with extensive performance metrics to confirm its reliability in real-world scenarios and various clinical conditions.

8. **Extensibility for Future Use Cases**
   Structuring the solution such that it can be extended to other medical image-based diagnostics beyond kidney-related abnormalities.

Impact and Outcome

- **Reduced Diagnosis Time:** Enables near-instant classification of CT scan images, helping healthcare professionals act faster.

- **Increased Accuracy:** CNN-based analysis reduces human error and improves early detection of kidney abnormalities.

- **Scalable Infrastructure:** With cloud-based deployment and containerization, the system can handle increased loads and serve multiple users concurrently.

- **Transparent and Reproducible Development:** Thanks to MLflow and DVC, every model update and dataset change is tracked, making audits and improvements efficient.

2. Motivation

The motivation behind this project stems from the growing need for faster, more accurate, and automated diagnostic tools in the field of medical imaging, specifically in the detection and classification of kidney diseases. Kidney disease is a global health issue, affecting millions of people worldwide. Early detection and timely treatment are crucial to prevent the progression of severe conditions such as chronic kidney disease (CKD) and kidney failure. However, the traditional methods of diagnosing kidney diseases, which primarily rely on the manual interpretation of medical images by radiologists, can be time-consuming and prone to human error. This creates an urgent need for automated systems that can assist healthcare professionals in improving diagnostic accuracy and efficiency.

The use of deep learning in medical image analysis has shown significant potential in various domains, including the detection of tumours, organ segmentation, and disease classification. The ability of Convolutional Neural Networks (CNNs) to automatically learn features from medical images makes them ideal for tasks such as kidney disease classification, where subtle differences in CT scan images can indicate different stages or types of disease. The success of deep learning models in other medical imaging applications, coupled with the increasing availability of large datasets and powerful computational resources, further motivates the development of an automated kidney disease classification system.

Another driving factor for this project is the shortage of skilled radiologists in many parts of the world, especially in rural and underdeveloped regions. This shortage results in delayed diagnoses and treatment, leading to worsened patient outcomes. By developing a system that can autonomously analyse CT scans and detect kidney diseases, this project aims to bridge the gap between the demand for radiological expertise and its availability. An automated system can serve as a valuable tool in regions where access to expert diagnosis is limited, ensuring that patients receive timely and accurate diagnoses.

Additionally, the integration of MLOps tools such as MLflow and DVC in the development process is motivated by the need for reproducibility and scalability in machine learning models. In many machine learning projects, ensuring that models can be consistently reproduced and scaled across different environments is a significant challenge.

The decision to deploy the system on AWS using Docker containers further supports the motivation for accessibility and scalability. With cloud-based deployment, healthcare providers across different regions and institutions can easily access and utilise the system without the need for significant infrastructure investments. This makes the technology more accessible to a broader audience and ensures that even smaller clinics or hospitals with limited resources can benefit from it.

Finally, the motivation for this project is driven by a desire to contribute to the broader goal of improving healthcare outcomes through technology. By combining deep learning with modern deployment techniques, the project aims to create a system that not only enhances diagnostic capabilities but also has the potential to save lives by facilitating early detection and treatment of kidney diseases.

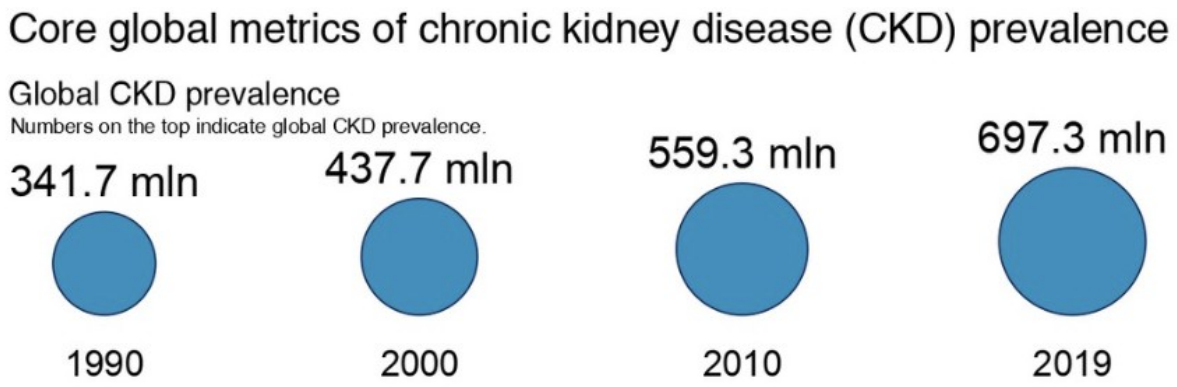

Fig 2.1 Global CKD Prevalence

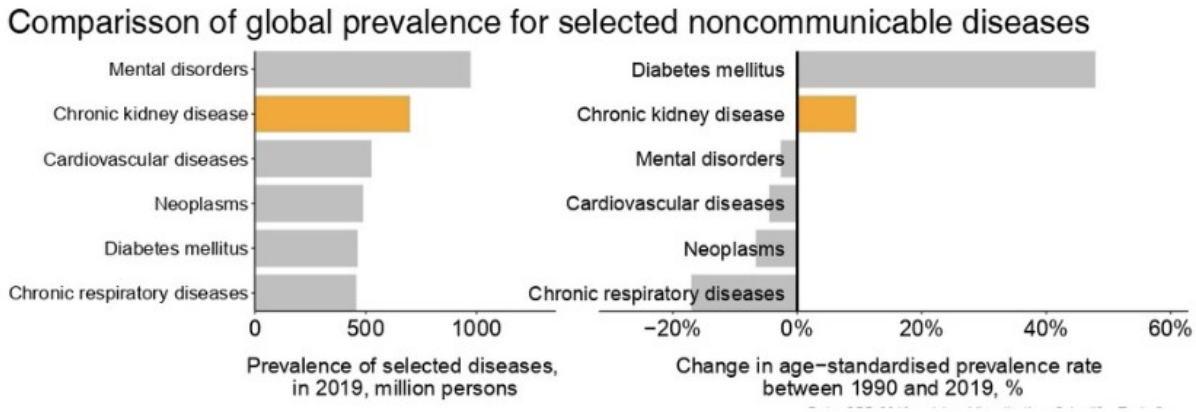

Fig 2.2 Comparison for noncommunicable disease

3. Literature Review

Kidney Tumour Detection and Classification Based on Deep Learning Approaches: A New Dataset in CT Scans

Kidney tumours present significant challenges in clinical diagnosis due to the complexity of CT scan interpretation, traditionally handled manually by radiologists. The study, published in the Journal of Healthcare Engineering via Wiley Online Library on October 22, 2022, addresses the critical need for an automated detection system that minimizes human error and reduces diagnosis time.

The study's authors introduce deep learning methodologies to enhance diagnostic accuracy and efficiency in identifying kidney tumours. Three key deep learning models—CNN-6, ResNet50, and VGG16—were applied to an extensive dataset comprising 8,400 CT scan images from 120 patients. These scans included both contrast-enhanced and non-contrast images to cover a broad spectrum of visual information and ensure robust training for the models. The CNN-4 model was specifically employed to classify tumours as malignant or benign, a feature crucial for timely therapeutic intervention.

Previous research on kidney tumour detection has relied heavily on traditional image processing techniques and classical machine learning, often constrained by limited accuracy and generalization capabilities. This study's literature review cites significant advancements in deep learning approaches, which outperform conventional methods by effectively identifying tumours in CT scans through feature extraction and pattern recognition. The methodology is based on existing successes in convolutional neural networks (CNNs), particularly in handling complex medical image data.

In this work, the authors implemented ResNet50 and VGG16, achieving detection accuracies of 97% and 96%, respectively. The CNN-4 model, specifically tailored for classification tasks, achieved a precision of 92% in distinguishing between malignant and benign tumours. These models showcase the potential of autonomous systems in enhancing clinical decision-making, underscoring the models' suitability for integration in diagnostic workflows to alleviate radiologists' workloads and improve diagnostic outcomes.

The study concludes that autonomous tumour detection models contribute to early-stage diagnosis, potentially improving patient outcomes by enabling prompt treatment. However, it emphasizes that the quality of the dataset is integral to the model's performance, indicating that diverse and larger datasets are necessary for further model validation and generalization across different patient populations.

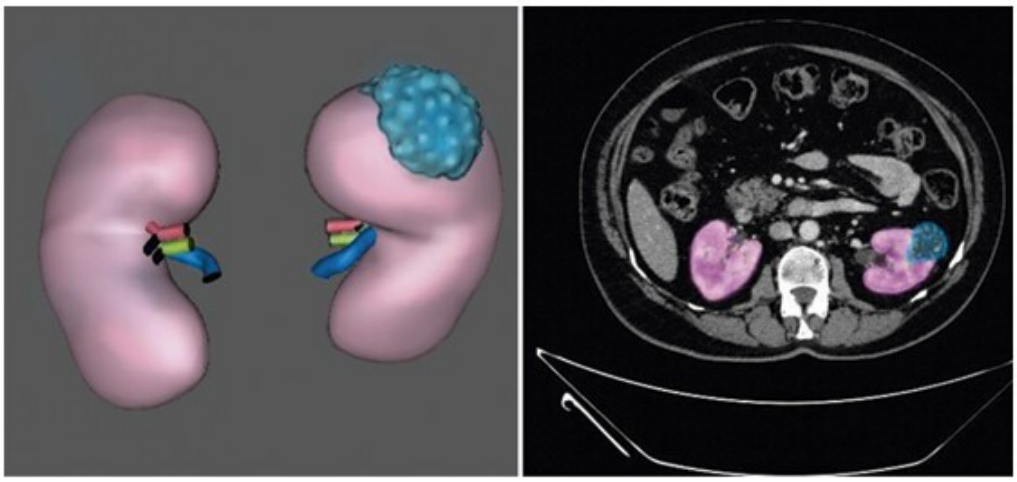

Fig 3.1 Sample Renal CT Scan

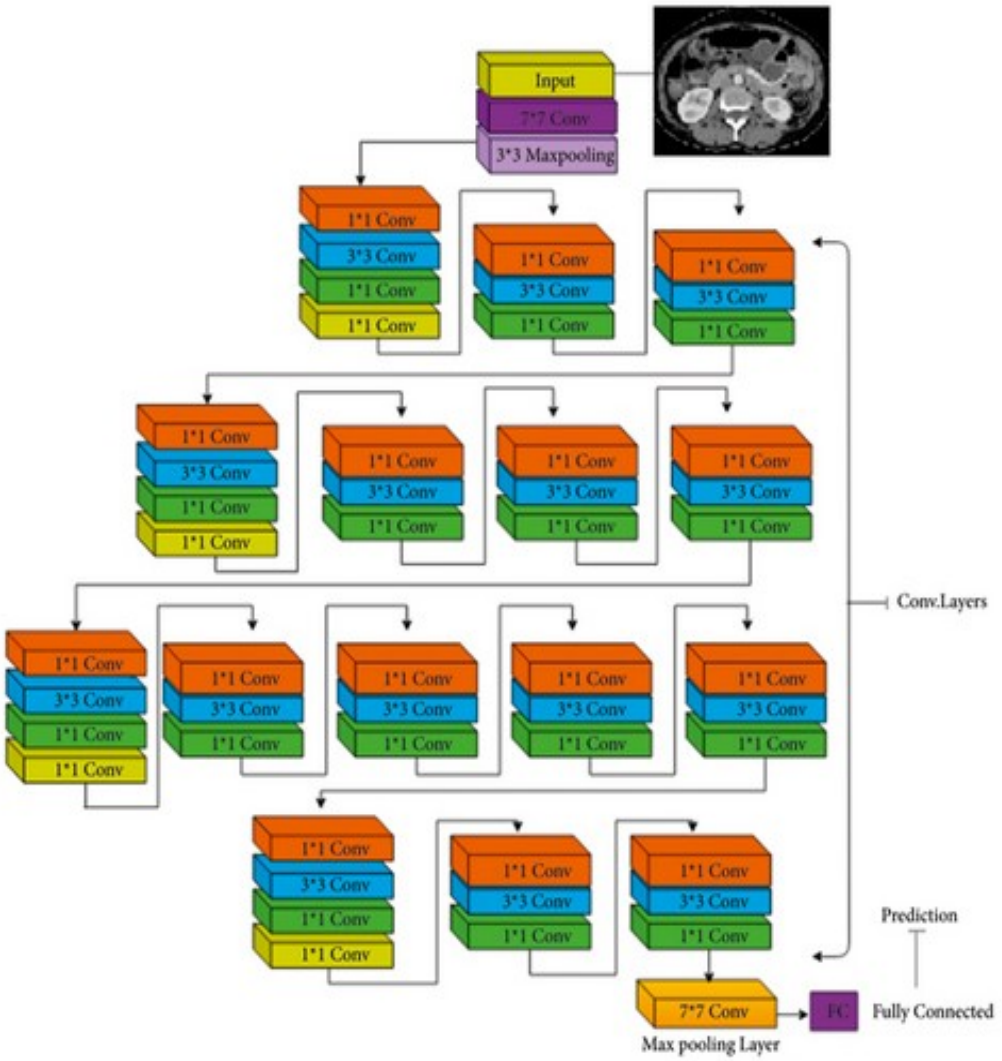

Fig 3.1 ResNET50 Architecture

Chronic Kidney Disease Prediction Using Machine Learning Techniques

Chronic Kidney Disease (CKD) is a progressive condition requiring accurate staging to guide treatment decisions effectively. Existing prediction models often employ binary classification, which is insufficient for clinical decision-making in managing CKD stages. Published in the *Journal of Big Data* in 2022, this study leverages machine learning techniques to predict CKD stages beyond binary classification, presenting a more nuanced approach to disease management.

This research utilizes patient data from St. Paulo's Hospital in Ethiopia, comprising 1,718 instances and 19 features, to develop predictive models for CKD stages. Key machine learning algorithms—Random Forest (RF), Support Vector Machine (SVM), and Decision Tree (DT)—were employed to predict CKD progression. The study incorporated Recursive Feature Elimination with Cross-Validation (RFECV) and ANOVA to select the most relevant features, enhancing the model's accuracy and efficiency in predicting CKD stages.

Building on previous studies in CKD prediction, which largely focus on binary outcomes, this research fills a gap by applying machine learning to multi-stage prediction. The authors' review of the literature indicates that while traditional models offer basic detection capabilities, machine learning models, particularly Random Forest and Support Vector Machine, have shown significant promise in predicting CKD stages with greater accuracy. Random Forest, combined with RFECV for feature selection, achieved an accuracy of 99.8% in overall CKD detection. When predicting specific CKD stages, Random Forest reached an accuracy of 79%, outperforming SVM and DT.

This study concludes that machine learning models, especially RF and SVM, have considerable potential in the early detection and staging of CKD. Such capabilities could be invaluable for clinical applications, particularly in resource-limited settings, by aiding healthcare providers in making timely and appropriate treatment decisions. However, the study acknowledges limitations, including the imbalanced dataset, which required resampling to achieve a balanced class distribution. Additionally, the relatively small sample size may restrict the model's generalizability across larger, more varied populations, indicating a need for further validation of diverse datasets.

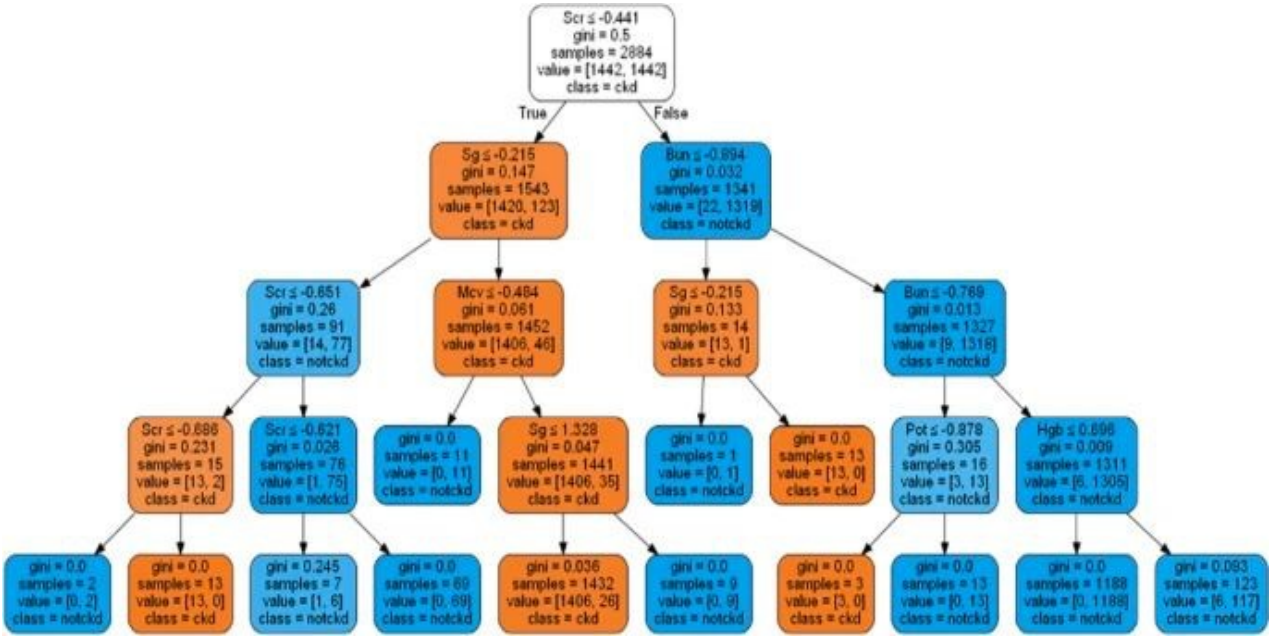

Fig 3.3 Model building flow diagram

Classification of Benign and Malignant Renal tumours Based on CT Scans and Clinical Data Using Machine Learning Methods

The accurate classification of renal tumours is critical for avoiding unnecessary surgical interventions, especially given that approximately 20% of renal masses ≤4 cm are benign. Conventional diagnostic methods relying on CT scans often lack precision in distinguishing between benign and malignant tumours, leading to overtreatment and increased patient burden. Published in *ResearchGate* and other AI healthcare journals in July 2023, this study explores machine learning-based approaches to address the limitations of traditional methods in renal tumour classification.

To achieve more accurate diagnoses, the study developed machine learning models using a dataset from 300 patients diagnosed with renal tumours. The model combined clinical data—such as demographic information and vital signs—with radiomic features derived from CT scans, providing a comprehensive approach to classification. By integrating multimodal data, the researchers were able to improve classification accuracy, advancing beyond the constraints of solely image-based or clinical data.

The literature review highlights previous approaches in renal tumour classification, which primarily utilized single-modal data. In contrast, the current study's multimodal framework demonstrates that combining clinical and radiomic data enhances model performance. Among the models tested, the Random Forest classifier achieved the highest precision, outperforming deep learning approaches. This model's results showed an AUC of 0.719, precision of 0.976, recall of 0.683, and specificity of 0.827, indicating its effectiveness in distinguishing benign from malignant tumours.

The study concludes that multimodal machine learning models significantly reduce overtreatment for benign tumours by increasing classification accuracy, thus allowing for timely intervention in malignant cases. This approach has the potential to improve clinical decision-making and patient outcomes in renal tumour management. However, the study acknowledges limitations, noting that external validation using independent datasets is necessary to ensure model generalizability across diverse clinical settings.

WHO 2022 Classification of Kidney tumours

The evolving classification of renal tumours reflects significant advances in molecular biology, prompting the need for revised diagnostic and treatment approaches in clinical practice. This literature, published as part of the *WHO 2022 Classification of Kidney tumours* and referenced in Caliò et al. (2023), discusses the impact of incorporating molecular insights into renal tumour classification, marking a departure from traditional cytology-based methods.

Historically, renal tumour classification relied primarily on cytological observations until molecular findings began to shape classification protocols in 1996. The *WHO 2022* classification incorporates new molecular entities such as TFEB- and TFE3-rearranged, SDH-, and FH-deficient renal cell carcinomas, all detectable through fluorescence in situ hybridization (FISH) assays. These refinements have also led to more precise definitions of existing tumour categories; for example, clear-cell papillary carcinoma is now differentiated from clear-cell carcinoma due to molecular distinctions.

The review of *WHO 2022* also notes the positive outcomes of this molecular approach in renal tumour classification. For hereditary cancers like FH- and SDH-deficient renal cancers, prognosis is now more accurately defined, improving patient outcomes. Additionally, the identification of ALK-rearranged tumours opens new avenues for targeted therapies, specifically through ALK inhibitors. Patient management has also advanced with this classification, allowing for enhanced genetic counselling and targeted surveillance, particularly valuable for families with hereditary cancer risks.

In conclusion, the *WHO 2022 Classification* exemplifies how molecular data integration has refined renal tumour diagnostics, facilitating better-targeted treatment and comprehensive management of hereditary cancers. Nonetheless, challenges remain. Molecular testing is resource-intensive and costly, which limits its accessibility in clinical settings globally, and there is a need for broader validation across diverse populations to ensure the robustness and generalizability of these findings.

Conclusion of Literature Review

Coarse review of the literature describes considerable improvements in diagnosis of kidney tumors and chronic kidney disease (CKD) using machine learning, deep learning and molecular classification techniques. Each study underscores a shared goal: In ocular disease, the goal is to improve diagnostic accuracy, reduce unnecessary treatments and improve patient outcomes.

By automating the task of tumor classification, Deep Learning models like ResNet50, VGG16 and CNNs demonstrate a great potential for detecting kidney tumours reducing the diagnostic time and human error. Similarly, Random Forest (RF) and Support Vector Machine (SVM) machine learning models, in the multi stage CKD prediction, outperform traditional binary classification methods and provide a fine grained understanding that is critical for clinical decision making. The integration of multi modal data (clinical and radiomic features) further increases diagnostic capabilities particularly for distinguishing benign from malignant renal tumors, for the purposes of treatment precision and resource allocation.

Furthermore, the WHO 2022 Classification of Kidney Tumors advances beyond a pathologic framework, including molecular data to enrich understanding of tumor pathology, risk stratification and targeted therapies. While this molecular approach has offered an advance of sorts in the management of hereditary cancers, this has been limited by high costs and elaborate validation.

Together, these studies provide the basis for greater accurate, efficient and personalized care in nephrology. The methodologies discussed have promising results, though limitations such as dataset quality, class imbalance, and the high resources expenses required for molecular testing point future research.

Aspect	Kidney Tumour Detection and Classification (2022)	Chronic Kidney Disease Prediction (2022)	Renal Tumour Classification via ML (2023)	WHO 2022 Classification of Kidney Tumours
Focus	Deep learning for detection and classification of kidney tumours in CT scans	Machine learning for multi-stage prediction of CKD	Machine learning using clinical + CT data for tumour classification	Molecular-based classification of renal tumours
Dataset	8,400 CT images from	1,718 instances with 19 features	Data from 300 patients including	Not a dataset-based study;

	120 patients	from St. Paulo's Hospital	radiomic + clinical data	literature-based classification update
Methods	CNN-6, CNN-4, ResNet50, VGG16	Random Forest, SVM, Decision Tree, RFECV, ANOVA	Random Forest, multimodal ML model (clinical + radiomic features)	Molecular assays (e.g., FISH), classification via genetic mutations
Performanc e	ResNet50: 97%, VGG16: 96%, CNN-4: 92% precision (malignant vs benign)	RF: 99.8% for CKD detection, 79% for staging	RF model: Precision 0.976, AUC 0.719, Specificity 0.827	Improved precision in subtyping, targeted therapy possible

Table 3.1 Comparison between Literature Reviews

4. System Requirements For Its Development And Production Environment.

For the successful development, testing, and deployment of the kidney disease classification system, several hardware, software, and cloud infrastructure requirements must be met. These requirements are essential for ensuring optimal model performance, seamless integration of MLOps tools, and smooth deployment in a production environment.

Hardware Requirements

Development Environment:

- **Processor:** A multi-core CPU with a minimum of 8 cores is recommended to handle complex computations during model training. We have an Intel Core i9 9900 Processor in our machine.
- **GPU:** A dedicated GPU with CUDA support is crucial for deep learning tasks. NVIDIA GPUs with at least 8GB of VRAM, are recommended for efficient model training and inference. We have a Nvidia RTX 2080 GPU in our machine.
- **RAM:** A minimum of 32GB of RAM is necessary to handle large datasets and to prevent bottlenecks during training.
- **Storage:** SSD storage with a capacity of at least 1TB is required to store CT scan images, datasets, and intermediate results generated during model training.
- **Network:** A stable, high-speed internet connection (minimum 100 Mbps) is essential for downloading pre-trained models, and libraries, and uploading datasets to cloud storage.

Production Environment:

- **Cloud Infrastructure:** The system will be deployed on Amazon Web Services (AWS), leveraging its Elastic Compute Cloud (EC2) instances.
- **Instance Type:** AWS EC2 instances equipped with NVIDIA Tesla V100 or A100 GPUs will be used for model inference in production, ensuring fast and efficient predictions.
- **Storage:** AWS S3 will be used for scalable storage of data and models, with a recommended storage of at least 100GB for initial deployment. S3 Glacier can be utilised for long-term storage of less frequently accessed data.
- **Load Balancing and Scaling:** AWS Elastic Load Balancing (ELB) and AWS Auto Scaling will be used to ensure the system can handle high traffic and automatically scale as needed.

Software Requirements

Development Environment:

- **Operating System:** The development environment will run on a Linux-based OS (preferably Ubuntu 20.04 or above) for better compatibility with deep learning libraries and tools.
- **Python:** Python 3.8+ is required, as it is the primary language for deep learning and machine learning model development.
- **Deep Learning Libraries:**
    - **TensorFlow or PyTorch** (latest versions) for model development and training.
    - **CUDA Toolkit and cuDNN** for enabling GPU acceleration in training deep learning models.
    - **OpenCV:** For image processing and manipulation of CT scan data.
    - **Pandas and NumPy:** For data manipulation and handling structured data.
- **MLOps Tools:**
    - **MLflow:** For experiment tracking, model versioning, and artefact management during the development phase.
    - **DVC (Data Version Control):** For versioning large datasets and ensuring consistency across different stages of model training.
- **Jupyter Notebook or PyCharm:** For development and experimentation in an interactive environment.

Production Environment:

- **Docker:** The system will be containerized using Docker to ensure consistency between the development and production environments. Docker containers will allow for easy scalability and deployment of the application across different platforms.
- **Kubernetes (Optional):** For large-scale deployments and orchestrating Docker containers, Kubernetes will be used to manage the containerized application.
- **AWS Tools:**
    - **AWS Lambda:** For serverless computing, handling small-scale tasks like preprocessing data or managing background jobs.

- **Amazon RDS (Relational Database Service):** For managing structured data related to patient records or model metadata.
- **AWS CloudWatch:** For monitoring system performance, managing logs, and setting up alerts in case of system failures or high resource usage.

Data Requirements

- **Dataset Size:** The project will require access to a large dataset of CT scan images, specifically focusing on kidney disease. A minimum of 10,000 images is recommended for training, validation, and testing. The dataset must be balanced to avoid bias in the model.
- **Data Format:** Medical image data should be in DICOM or NIfTI format for high-resolution CT scan images. These formats provide rich metadata, which is crucial for model training and evaluation.
- **Data Preprocessing:** Preprocessing will involve resizing, normalisation, and augmentation of CT scan images to ensure they are suitable for training the CNN model.

Security and Compliance Requirements

Since the system will handle medical data, it must adhere to security protocols and compliance regulations such as HIPAA (Health Insurance Portability and Accountability Act) in the United States or GDPR (General Data Protection Regulation) in Europe. Some key security measures include:

- **Data Encryption:** Encrypt sensitive patient data both at rest and in transit using AES-256 encryption.
- **Access Control:** Implement strict access control policies, ensuring that only authorised personnel can access the system and data.
- **Audit Logs:** Maintain detailed logs of all actions performed within the system for compliance and auditing purposes.

Maintenance and Scalability

- **Continuous Integration/Continuous Deployment (CI/CD):** Implement CI/CD pipelines using tools like Jenkins or GitLab CI to automate testing, integration, and deployment.
- **Monitoring and Logging:** Set up monitoring tools like Prometheus and Grafana for real-time system health tracking and performance monitoring.

- **Backup and Disaster Recovery:** Implement automated backup strategies using AWS Backup to ensure that critical data is securely stored and easily recoverable in case of failures.

5. Stakeholders

The successful execution of any project requires collaboration and input from various stakeholders who play pivotal roles throughout the project's lifecycle. This kidney disease classification project, which utilises deep learning on CT scan images, involves multiple stakeholders, each contributing in different capacities. Understanding their roles and responsibilities ensures clear communication, smooth workflow, and alignment with the project's objectives.

Primary Stakeholders

1. Healthcare Professionals: Healthcare professionals, particularly nephrologists, radiologists, and general physicians, are key stakeholders in this project. Their expertise in kidney diseases and medical imaging is crucial for validating the model's accuracy and utility in a real-world clinical setting. They provide the medical knowledge required to interpret CT scan images and help in labelling the dataset during the data preparation phase. Moreover, they are the end-users of the system who will leverage the model's predictions to make diagnostic decisions. Their feedback is essential for iterative improvements in the system, ensuring that the model aligns with the diagnostic needs and workflows of medical practitioners.

2. Patients: Although patients are not directly involved in the development of the project, they are the ultimate beneficiaries. The system aims to improve diagnostic accuracy and early detection of kidney diseases, leading to timely interventions and better patient outcomes. Their medical data (CT scan images) is used to train and validate the model, and thus their privacy and data security must be safeguarded in compliance with regulations like HIPAA or GDPR. In this sense, patients indirectly shape the ethical and legal considerations of the project.

3. Data Scientists and AI/ML Engineers: Data scientists and machine learning engineers are responsible for developing the deep learning model and fine-tuning it to ensure optimal performance. They handle tasks such as data preprocessing, feature extraction, model training, and evaluation. Additionally, they ensure that the system integrates with MLOps tools like MLflow for experiment tracking and DVC for data versioning, which enhances model reproducibility and scalability. Their role also extends to maintaining the system in production, troubleshooting issues, and refining the model based on real-time performance metrics.

4. Software Developers: Software developers are integral to the project's architecture and deployment. They are responsible for building the application's backend, integrating the trained model into the system, and ensuring that it can be accessed in clinical settings through

a user-friendly interface. They work closely with cloud engineers to ensure that the system runs efficiently on cloud platforms such as AWS, leveraging services like Docker for containerization and EC2 for scalable compute resources. Their expertise ensures that the system remains functional, scalable, and secure in both development and production environments.

Secondary Stakeholders

1. Hospital Administrators: Hospital administrators are crucial decision-makers when it comes to adopting new technologies in healthcare settings. They are responsible for assessing the cost-effectiveness, scalability, and overall impact of the deep learning system on clinical operations. Their approval is required for deploying the model in hospitals and clinics. They also oversee the procurement of necessary hardware and cloud resources for the system's operation. Additionally, hospital administrators are responsible for ensuring that the system complies with medical regulations and standards.

2. Cloud Infrastructure Providers: Cloud infrastructure providers like AWS play a vital role in supporting the deployment and scaling of the deep learning system. AWS provides compute power (EC2 instances with GPU support), data storage (S3), and other essential services such as AWS Lambda for serverless computing. These cloud services ensure that the system is available to healthcare professionals on demand and that it can scale efficiently in response to varying workloads. The cloud provider also ensures data security through encryption and access control, making them a key technical stakeholder.

3. Regulatory Bodies: Regulatory bodies such as the Health Insurance Portability and Accountability Act (HIPAA) in the United States or the General Data Protection Regulation (GDPR) in Europe play a significant role in overseeing the legal compliance of the project. These organisations set the standards for data privacy, patient consent, and medical device regulations, which the project must adhere to. Ensuring that the model meets these legal requirements is crucial for its deployment in healthcare settings. Failure to comply with these regulations can lead to legal challenges and compromise the project's success.

Tertiary Stakeholders

1. Investors and Funding Agencies: Investors or funding agencies, such as government healthcare grants or private health-tech investors, may have a vested interest in the development of this system. They provide the financial resources required for model development, data collection, cloud infrastructure, and the deployment of the system. These stakeholders are primarily concerned with the return on investment (ROI) and the project's potential for commercialization and scalability. Their support can be pivotal in taking the project beyond the development phase and into widespread use.

2. Technical Support Teams: Technical support teams, including IT professionals, ensure that the system runs smoothly post-deployment. They handle issues related to software maintenance, server management, and network performance. Their role also involves ensuring that the system remains operational with minimal downtime and that any bugs or security vulnerabilities are addressed promptly. In the event of system malfunctions, technical support teams are the first responders to mitigate risks and ensure continuity.

Users in Research and Academia

1. Researchers: Researchers in the field of medical AI and image analysis are secondary users of this system. They may utilise the system's framework for further research into kidney disease classification or adapt the model architecture for other types of medical imaging applications. Researchers contribute to validating and improving the model by providing peer reviews and publishing studies based on its effectiveness. Their involvement adds academic credibility and helps in knowledge dissemination within the medical and AI communities.

6. Project Design/Data Flow Diagram / UML

Overview

The goal of this project is automated classification of kidney diseases by using the CT scan images using deep learning. The objective of this system is to leverage Convolutional Neural Networks (CNNs) applied to medical imaging data with the purpose to increase of diagnostic accuracy, and decrease the burden on healthcare providers. The project follows the modern MLOps practices of tracking experiments with MLflow and data versioning with DVC (Data Version Control), letting them be reproduced and kept stable during model development. It runs as Docker containers on AWS cloud infrastructure and is very easily scalable and accessible to various sizes of the healthcare facility.

Data preprocessing, model development, training, evaluation components, followed by a user friendly web interface to allow medical professionals in uploading and processing CT scans for analysis are key components of the thesis. In this project, we aim to combine the best of state of the art deep learning techniques and practical deployment strategies to help improve the efficiency and effectiveness of kidney disease diagnosis in real clinical settings.

Gantt Chart / Timeline

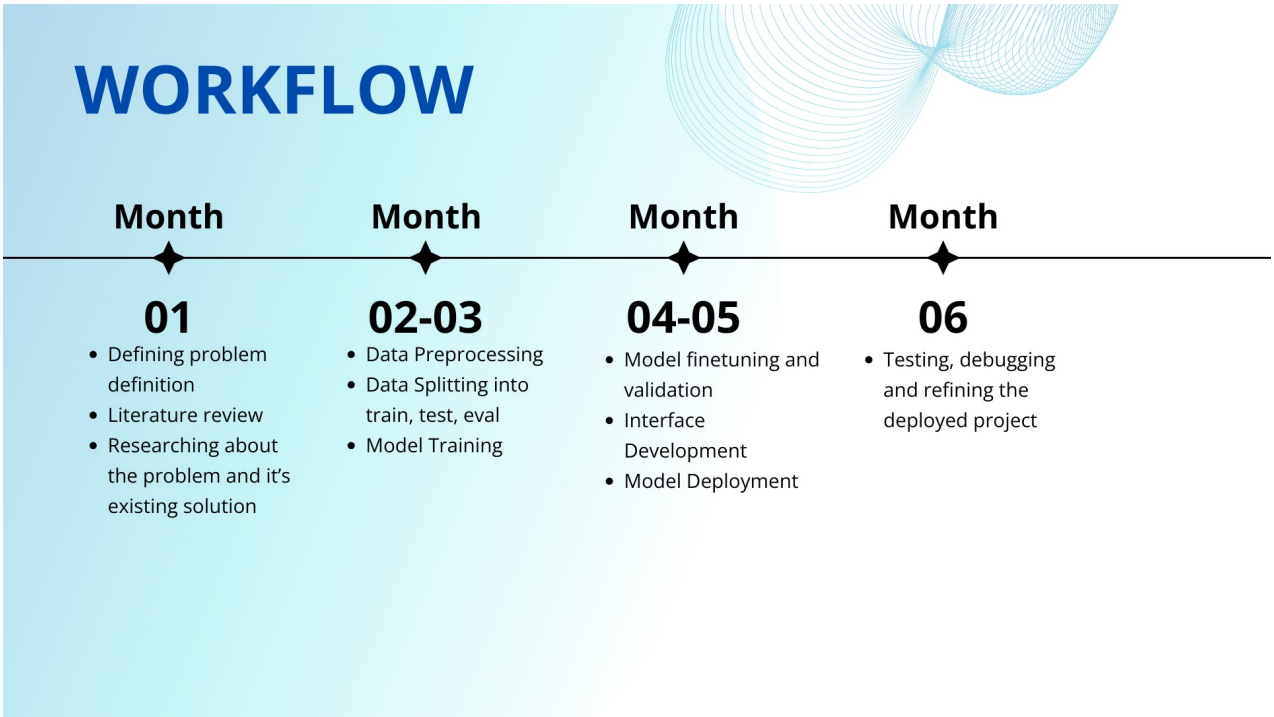

Fig 6.1 Gantt Chart/Timeline

UML

The UML diagrams (primarily **Use Case**, **Class**, and **Sequence Diagrams**) help model the structural and behavioral components of the system. The **Use Case Diagram** showcases interactions between the primary actor (medical professional) and the system functionalities, such as uploading scans and receiving diagnostic results. The **Class Diagram** defines the major components—like ImageProcessor, ModelPredictor, UserInterface, and Logger—and their interactions. The **Sequence Diagram** maps the order of operations, starting from the image upload, going through preprocessing and classification, and ending with results being displayed. These UML diagrams provide a clear architectural blueprint of the system and help in understanding both the high-level design and the workflow between components.

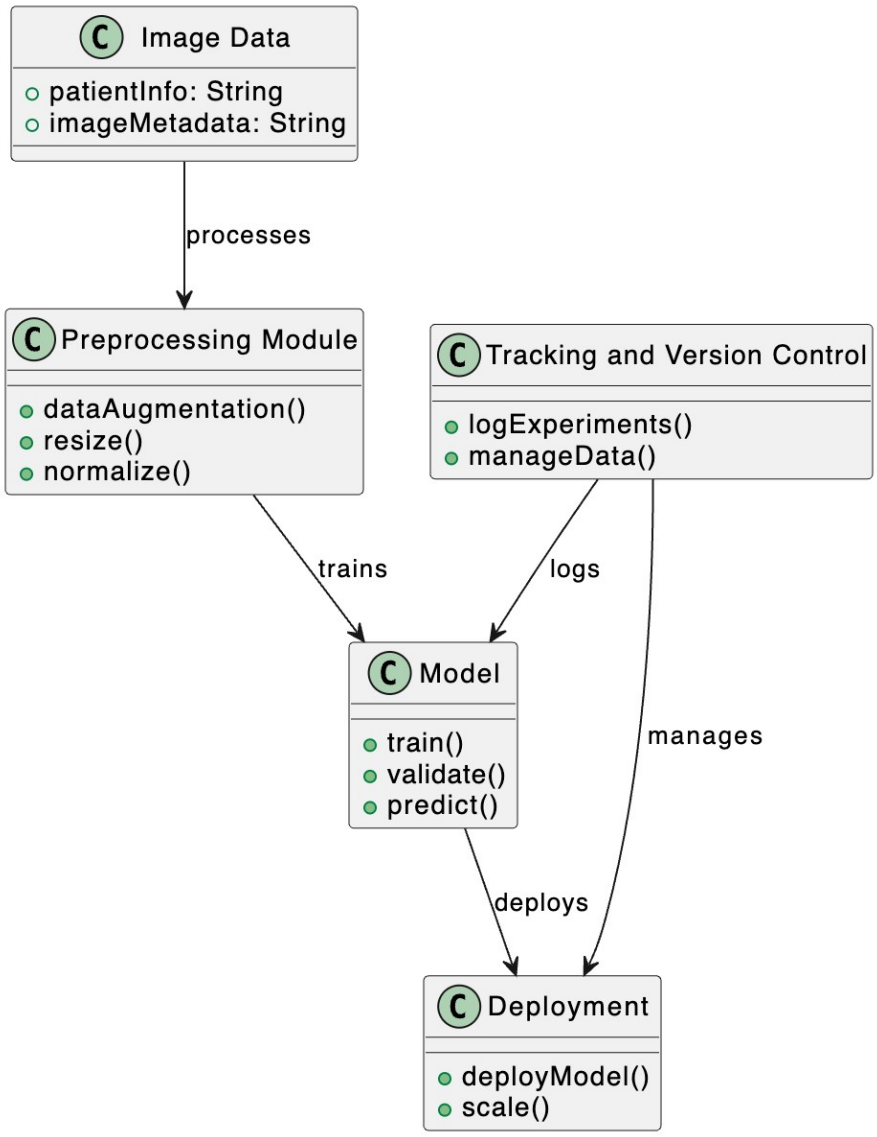

Fig 6.2 UML Diagram

Data Flow Diagram

The Data Flow Diagram (DFD) illustrates the movement of data through the kidney disease classification system. It highlights how raw CT scan images are uploaded by the user and then passed through various processing stages including preprocessing, model inference, and result delivery. The flow begins with the **user uploading CT scan images** via a web interface. These images are then passed to the **preprocessing module**, which normalizes and resizes them. The processed images are sent to the **CNN-based model**, which classifies them into categories based on potential kidney diseases. The **prediction results** are then displayed to the user through the web interface. Supporting components like MLflow and DVC operate in the background to track model metrics and manage data versions, while Docker and AWS ensure seamless cloud-based deployment.

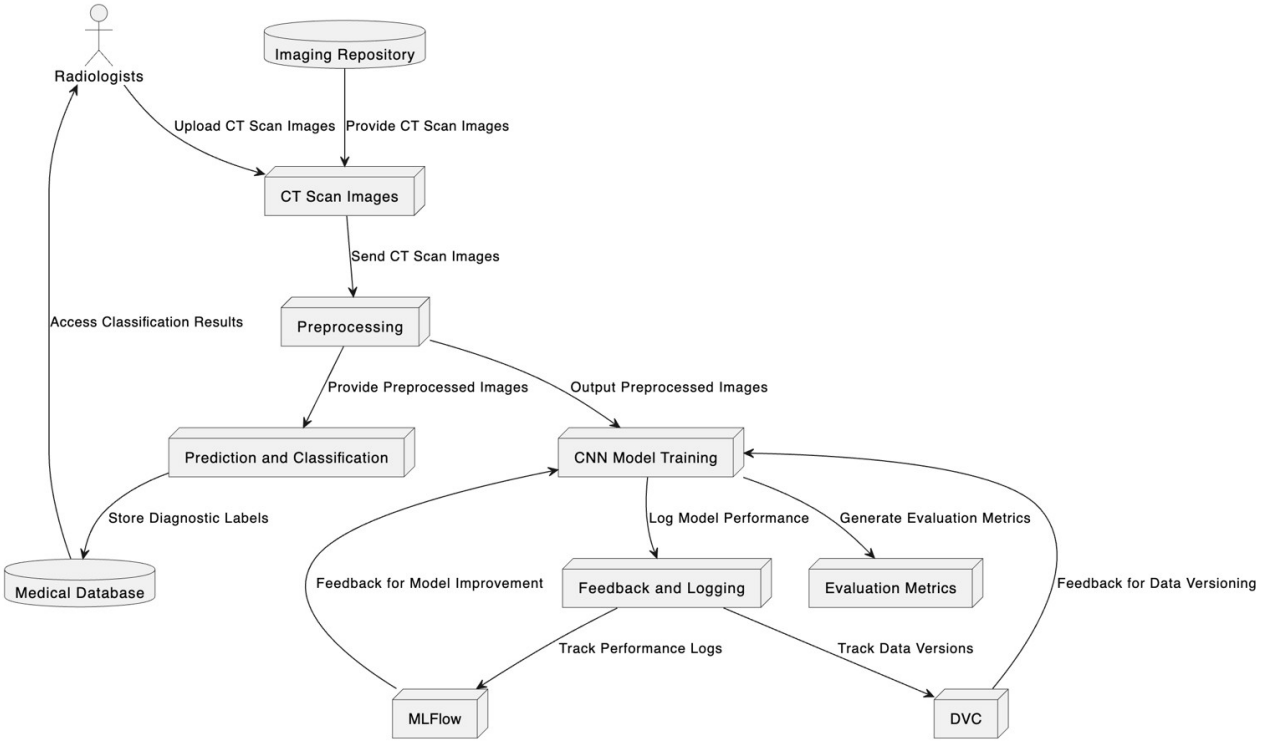

Fig 6.3 Dataflow Diagram

7. Approach Used

7.1 GitHub Repository Setup

Setting up a GitHub repository is essential for managing code, collaborating, and maintaining a version-controlled backup of the project. For this project, a repository named "Kidney Disease Classification Deep Learning Project" was created to store all code and related resources.

The repository setup involved the following steps:

1. **Creating the Repository:**
    - Go to GitHub and click on “New” under the repositories section.
    - Name the repository (e.g., "kidney-disease-classification-deep-learning-project").
    - Select options like adding a README file to describe the project, then click “Create repository.”
    - Once created, options to clone the repository appear.
2. **Cloning the Repository Locally:**
    - Copy the repository's HTTP link from GitHub by clicking on the "Code" button.
    - Open a terminal (e.g., Git Bash or Anaconda Prompt) in the folder where you want to store the project.
    - Run the command:
        i. git clone <repository-link>
    - This clones the repository into the specified local folder.
3. **Accessing the Repository Locally:**
    - After cloning, navigate to the project folder using:
        i. cd kidney-disease-classification-deep-learning-project
    - This command ensures you are working inside the project directory.
4. **Opening the Repository in a Code Editor:**
    - Open a code editor like Visual Studio Code (VS Code) or PyCharm. To launch VS Code directly from the terminal, use:

i. code .

5. **Configuring VS Code:**

    - In VS Code, close any unnecessary windows or welcome screens to focus on organising the project structure and writing code.

Following these steps, the project repository is set up for version control and collaboration, ensuring the code remains secure, backed up, and easily shareable.

7.2 Project Template Creation

To efficiently set up the project folder structure, we avoid the time-consuming task of manually creating directories and files. Instead, a Python script automates this setup, making it a one-time effort that ensures consistency, reusability, and efficiency.

Step-by-Step Guide

1. **Create a Python Script**: A Python script, template.py, is created to handle the entire folder structure setup. Using Python's os and pathlib modules, the script generates folders and files, while the logging module tracks and logs each action for clear feedback.

2. **Logging Setup**: A logging system provides real-time updates on folder and file creation. Instead of printing information to the terminal, this system logs each action with time stamps for effective tracking.

3. **Define Project Name**: A default project name, such as CNN_Classifier, is set in the script to establish the folder structure base. This can be modified to suit different project needs.

4. **Folder and File Structure**: The script defines key directories and files, creating the following structure:

    - .github/workflows/ – for GitHub Actions (CI/CD pipeline).

    - src/ – main source code directory with subfolders for components like data, training, and models.

    - utils/ – for utility functions.

    - config/, constants/, pipeline/ – for configuration files, constants, and pipeline code.

    - templates/ – for Flask web app files, including an index.html file.

    - requirements.txt – lists dependencies.

    - setup.py – script for project setup.

5. **File and Folder Creation Logic**: Using a loop, the script iterates over the list of directories and files, creating them automatically. The use of Python's pathlib.Path class ensures compatibility across various operating systems (Windows, macOS, Linux) by handling system-specific path separators.

6. **Execution**: Run the script with the following command in the terminal:

    - python template.py (This command generates the defined files and folders.)

7. **GitHub Integration**: After generating the folder structure, commit the changes to GitHub (This process ensures that the folder structure is version-controlled and accessible via the GitHub repository.):

    - git add .
    - git commit -m "Added folder structure"
    - git push origin main

Using this automated setup, we save time, reduce errors, and ensure that a standardised folder structure is available for future projects.

7.3 Requirements Installation & Project Setup

To set up the environment and install the necessary dependencies for this project, follow these steps:

1. **Create a Virtual Environment**:
   Start by creating a virtual environment using conda to keep dependencies isolated and ensure smooth project execution.
   conda create --name <environment_name> python=3.8 -y
   For this project, the environment is named "kidney." You can specify any name:
   conda create --name kidney python=3.8 -y

2. **Activate the Environment**:
   Once the virtual environment is created, activate it:
   conda activate <environment_name>
   For example, activate it here with:
   conda activate kidney

3. **Install Required Packages**:
   Install all dependencies listed in the requirements.txt file with:
   pip install -r requirements.txt
   Key packages include:

- tensorflow==2.2.0: For deep learning operations.
- Pandas, numpy, matplotlib, seaborn: Libraries for data manipulation and visualisation.
- dvc: For Data Version Control.
- mlflow==2.2.2: For experiment tracking and model management.
- pyyaml: For parsing YAML configuration files.
- Flask: To build the web application.
- gdown: For downloading data from Google Drive.

4. **Setting Up Local Package**:
   Install the local project folder as a package to enable imports for modules like SRC/CNN_classifier using setup.py:
   pip install -e .

5. **Cloning the Repository**:
   Clone the project repository to set up the environment:
   git clone <repository_link>

6. **Committing and Pushing Changes**:
   Use VS Code's source control feature to commit and push changes to GitHub. After making updates, commit with a descriptive message like "requirements added" and push directly from VS Code.

After these steps, your project environment is ready for implementation.

7.4 Logging, Exception Handling & Utilities Modules

The logging, exception handling, and utility modules are foundational components in building a well-structured and scalable application, especially in deep learning projects. Here’s how each module contributes to the overall project architecture:

Logging Module

The logging module enables tracking the code's execution flow, which is invaluable for debugging and monitoring purposes. For this project, a custom logging setup was implemented using Python’s built-in logging package, configured to capture logs both in a dedicated log file and in the terminal. Here’s how it works:

- **Initialization**: A centralised logger is defined within the project's __init__.py file, allowing easy import across multiple components without repeatedly defining logging configurations.

- **Log Format**: The logging format includes a timestamp, log level, module name, and message. This format ensures that the log records are detailed and self-explanatory, making it easier to trace issues.

- **Log File and Terminal Output**: Logs are saved in a logs folder as well as printed in the terminal using sys.stdout. This setup ensures that log data is accessible both during development and in production, where direct terminal access might be unavailable. The log file enables offline inspection of errors and application states post-deployment.

Exception Handling Module

To manage errors effectively, a custom exception-handling module is used. It leverages the python-box library, specifically the Box class, to wrap exceptions in a standardised format. Key aspects include:

- **Box Exceptions**: With Box exceptions, errors are encapsulated, allowing for flexible data handling in nested structures, which enhances readability when exceptions are logged.

- **Common Error Information**: The exception module logs relevant information such as error type, location, and error message. This makes debugging efficient by pinpointing exactly where and why failures occur.

Utils Module

The utils module hosts a suite of common functions used across the project, reducing redundancy and improving modularity. Key utility functions include:

- **File Operations**: Functions for saving/loading JSON and binary files, reading YAML configurations, and handling base64 image encoding/decoding. These are vital for data processing and configuration management.

- **Directory Management**: A helper function to create directories as needed. This is particularly useful in machine learning pipelines, where new folders are often generated for model artefacts, logs, and checkpoints.

- **Config Box Type**: Instead of returning simple Python dictionaries, config data from YAML files is returned as a ConfigBox (from the python-box library). This approach enables accessing dictionary keys as attributes (e.g., config.key rather than config['key']), making the code cleaner and more intuitive.

Together, these modules form a robust foundation, ensuring that logging, error handling, and utility operations are managed efficiently throughout the project, contributing to a maintainable and scalable codebase.

7.5 Project Workflows

This project follows a structured workflow, progressing through a series of steps to update files and implement components. Below is an outline of the workflow:

1. **Update config.yaml**:
   Begin by updating config.yaml, which includes crucial settings and parameters for the project. If sensitive information like database credentials is needed, it would be stored in a separate secrets.yaml file (though this project does not use one).

2. **Modify params.yaml**:
   Next, update the params.yaml file, which holds model-specific parameters essential for fine-tuning performance.

3. **Update Entity Folder**:
   The Entity folder in SRC defines data structures utilized throughout the project. Configure these structures before progressing to other components.

4. **Configure the Configuration Manager**:
   Update configuration.py in the config folder within SRC. This file manages configurations centrally, making it essential to update early on to allow other parts of the project to reference settings effectively.

5. **Set Up Components**:
   Configure individual modules, like data ingestion, model preparation, model training, and model evaluation, located in the components folder in SRC. These modules handle data processing, model building, and evaluation.

6. **Create and Configure Pipelines**:
   In the pipeline folder, set up both the training and prediction workflows, which manage the model's training and deployment processes.

7. **Finalise main.py**:
   Update main.py, the primary entry point for the project, towards the end. This file connects various modules and endpoints within the project.

8. **Update dvc.yaml**:
   Update dvc.yaml to track data and model versions using Data Version Control (DVC), monitoring changes in the pipeline.

9. **Set Up app.py**:
   Finally, configure app.py, which defines the web application's routes and endpoints, enabling user interaction with the model.

Following these steps ensures a progressive project setup, verifying that each component integrates smoothly within the overall architecture and facilitating efficient implementation and deployment.

7.6 Data Ingestion Component

The **Data Ingestion Component** is the first and one of the most critical stages of the machine learning pipeline, responsible for retrieving raw data and organizing it in a form suitable for downstream processing. In this project, the ingestion pipeline has been designed with robustness, modularity, and scalability in mind, ensuring that the data is not only retrieved correctly but also structured appropriately for training and evaluation purposes.

Objective

The primary goal of the data ingestion component is to:

- Retrieve kidney CT scan image data from a cloud-based source (Google Drive).
- Automatically download, extract, and organize the dataset.
- Maintain consistency and reproducibility using configurable parameters.
- Facilitate easy integration with other pipeline stages such as preprocessing, model training, and evaluation.

Dataset Description

The dataset used for this project consists of **CT scan images of kidneys**, specifically curated to include both *normal* and *tumour-affected* kidney samples. The dataset is intentionally kept small for demonstration purposes, containing:

- **225 images of normal kidneys**
- **A limited number of tumour-affected kidney images (around 15)**

Due to the small size (~240 images total), this dataset is sufficient to showcase the functionality of the pipeline and model but is acknowledged to be limited in terms of enabling robust model generalization.

To facilitate accessibility and sharing, the dataset has been **compressed into a ZIP file and uploaded to Google Drive**. This approach allows the data to be retrieved dynamically at runtime, avoiding the need to store large datasets locally or in a Git-based version control system (such as GitHub or GitLab).

Data Ingestion Workflow Overview

The ingestion process involves multiple well-defined steps:

1. **Configuration Setup**
   Custom data classes are used to define configuration entities. These include key variables such as:
   - The dataset's download URL (Google Drive link).
   - Local paths for storing the ZIP file.
   - Directory locations for extracted data.
   - File naming conventions and folder structures.
2. This approach promotes reusability and makes the pipeline adaptable to future changes (e.g., using a different dataset or directory structure) without modifying core logic.

3. **Class Definition: `DataIngestion`**

   The ingestion logic is encapsulated in a class named `DataIngestion`, which receives configuration parameters from a centralized **Configuration Manager**. This class contains two major methods:

   - **`download_data()`**:

     Utilizes the `gdown` Python package to download the dataset from the specified Google Drive link. The function:

     - Verifies whether the target download directory exists; if not, it creates it.
     - Constructs the correct download path and filename.
     - Retrieves the ZIP file using `gdown.download()`.
     - Logs all operations for traceability and debugging.

   - **`extract_zip_file()`**:

     Handles the unzipping of the downloaded dataset using Python's built-in `zipfile` module. This method:

     - Opens the ZIP file.
     - Extracts its contents into the specified directory structure.
     - Ensures that files are placed under properly named folders (e.g., `/normal/`, `/tumor/`).
     - Logs progress and any file-related issues that might occur during extraction.

4. **Pipeline Execution**

   The ingestion process is structured as a two-step **pipeline** to ensure modular execution and error handling. The pipeline consists of:

   - **Step 1: Download Data**

     - The `download_data()` function is called.

        - If the file already exists locally, the function can skip re-downloading to avoid redundancy.
        - A log entry confirms successful download or indicates an issue if any occurs.

    - **Step 2: Extract Zip File**
        - The `extract_zip_file()` function is executed.
        - Data is unzipped and organized as per the directory structure expected by downstream processes (e.g., `/data/normal`, `/data/tumor`).

    - The entire pipeline is wrapped in a **try-except block** to catch exceptions and provide clear error messages in the logs. This prevents pipeline crashes and supports debugging in case of failure.

7.7 Prepare Base Model Component

The **Prepare Base Model Component** is a crucial step in the deep learning pipeline, responsible for configuring and saving a foundational model tailored to the kidney CT image classification task. This component utilizes **Transfer Learning**, a technique that adapts a pre-trained model (e.g., VGG16, ResNet50, or InceptionV3) to a new task by fine-tuning the architecture. By leveraging features already learned from large datasets like **ImageNet**, we significantly reduce the computational cost and training time while improving model performance, especially when the available dataset is limited.

Workflow Overview:

1. **Downloading the Pre-trained Model:**

- The pipeline begins by **loading a pre-trained convolutional neural network (CNN)**, such as VGG16, using Keras Applications.
- These models are trained on the **ImageNet dataset**, which contains over a million images across 1000 classes, and are well-suited for feature extraction due to their deep architectures.

- The model is loaded with `include_top=False`, which excludes the fully connected (FC) layers responsible for ImageNet classification. This allows us to attach our own task-specific layers.

- The downloaded base model is typically stored in **HDF5 format (.h5)** for further modifications and integration into the training pipeline.

- The pre-trained convolutional base helps capture low-level features such as edges, textures, and object parts, which are transferable across various computer vision tasks.

2. Customising the Model:

- After removing the top (FC) layers of the pre-trained model, we **add a new classification head** tailored to our specific task.

- In our project, we are dealing with a **binary classification** problem (e.g., normal vs. abnormal kidney CT images).

- The custom top layers include:

    - A **Global Average Pooling** layer to reduce the dimensionality of the feature maps from the last convolutional layer.

    - One or more **Dense (fully connected) layers**, possibly with Dropout for regularization.

    - A **final Dense layer with 2 neurons** (representing two classes) and a **Softmax activation function** to output class probabilities.

- This structure ensures that while the convolutional base captures general features, the FC layers focus on learning task-specific patterns.

3. Freezing Layers:

- **Freezing** layers means setting `trainable=False` on certain layers to prevent their weights from being updated during backpropagation.

- In this step:

    - All or a subset of the convolutional layers in the base model are frozen.

    - Freezing is particularly useful when working with a small dataset, as it prevents overfitting and retains the robust features learned from ImageNet.

- The flexibility to **selectively freeze layers** is offered based on:

    - Dataset size (smaller datasets benefit more from freezing).

    - Computational resources.

    - Desired level of feature reuse vs. task-specific fine-tuning.

- In scenarios where we have a larger dataset or want more task-specific learning, we may choose to **fine-tune deeper layers** of the base model.

4. Saving the Customised Model:

- Once the model is customised, it is compiled with:

    1. A **loss function** like `categorical_crossentropy` or `binary_crossentropy`.

    2. An **optimizer** such as Adam with a specified learning rate.

    3. **Metrics** like accuracy to monitor performance.

- The model is then saved in two stages:

1. The **original base model** (pre-trained without modifications) is saved for reference or potential reuse.

2. The **updated model**, now equipped with custom classification layers, is saved for training.

- These models are stored in the **artifacts directory** under structured folders, promoting modularity and traceability.

Key Configurations:

- **Input Shape**: Specifies the shape of input images expected by the model. For VGG16, this is usually (224, 224, 3), meaning RGB images of 224x224 resolution.

- **Classes**: Indicates the number of output nodes in the final Dense layer. For binary classification, this is set to 2.

- **Learning Rate**: A small float (e.g., 0.0001) that controls how much the model weights are updated during training.

- **Include Top**: By setting `include_top=False`, we discard the original FC layers of the pre-trained model, enabling customisation.

Code Walkthrough:

Configuration Management:

- The model parameters such as input shape, number of classes, learning rate, etc., are stored in a **YAML configuration file** for easy access and modification.

- A configuration manager class is responsible for **loading these parameters** and making them available to the model preparation component.

- This design enhances **modularity, reusability, and reproducibility** of the code.

Base Model Preparation:

- A dedicated class named `PrepareBaseModel` is created to encapsulate all functionalities required to set up the base model.

- The class consists of:

    - A method `get_base_model()` that fetches the pre-trained model architecture.

    - A method `update_base_model()` that adds custom FC layers and applies freezing as required.

    - A method `save_model()` that stores both the base and customised models.

- This structure ensures that all actions related to model setup are **encapsulated, testable, and extendable**.

Saving the Model:

- After finalizing the architecture, both the **unmodified base model** and the **customised model** with classification layers are saved using Keras's `model.save()` method.

- The models are saved into the **`artifacts/prepare_base_model/`** folder with proper naming conventions.

- This ensures that the model can be directly loaded in the **Model Trainer Component** without any architectural changes or redefinition.

This component lays the **foundation for transfer learning**, allowing rapid development of high-performance models with limited data and compute. By effectively combining a robust pre-trained base with custom top layers, the system is optimized for real-world deployment in medical image classification tasks such as kidney CT diagnosis.

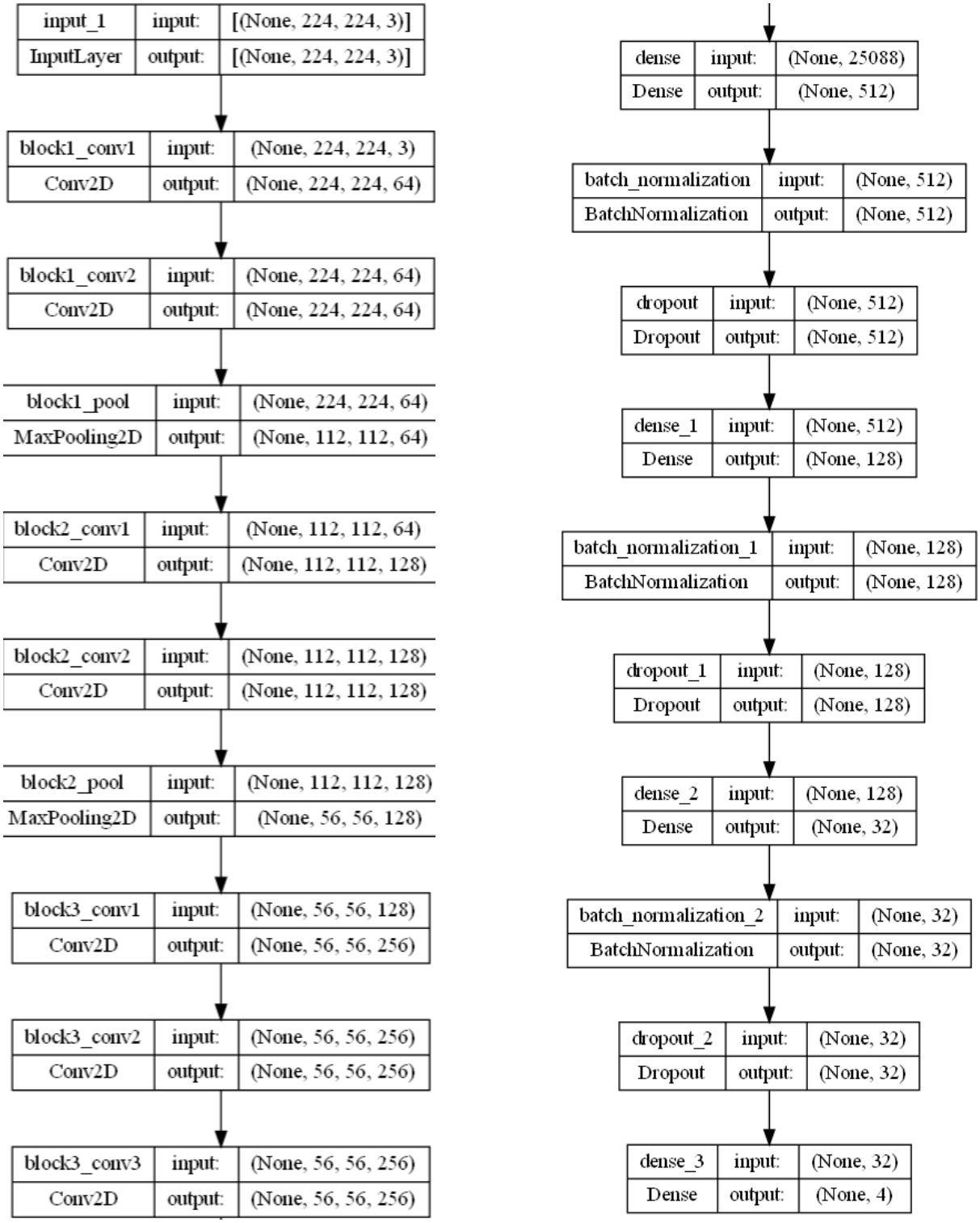

Fig 7.1 Base Model

7.8 Model Trainer Component

The Model Trainer Component is responsible for orchestrating the training phase of the deep learning pipeline. It utilizes the preprocessed data and the updated base model to fine-tune parameters and optimize the model's performance. This component ensures a structured, modular, and reproducible training process that supports dynamic configurations, efficient data handling, and robust model management.

1. Model Training Configuration

The training configuration is centralized within a YAML or JSON configuration file, ensuring clarity, reusability, and adaptability. It includes essential hyperparameters and directory paths, such as:

- Model Save Path: Specifies the destination within the artefact directory for storing the final trained model.
- Epochs and Batch Size: Defines how many times the model will iterate over the training data and how many samples it will process per iteration.
- Image Parameters: Sets the input image dimensions, rescaling factor, and class mode.
- Data Augmentation Flags: Enables real-time augmentation to artificially expand the dataset size and enhance generalization.
- Validation Split Ratio: Typically set to 0.2 (80-20 split) to ensure a sufficient amount of validation data.

All configurations are dynamically adapted based on the project root, making the training pipeline easily portable across different environments.

2. Loading the Base Model

The get_base_model() function is used to load the fine-tuned base model from the artefact directory. This model was previously updated with a custom classification head during the preparation phase.

- Model Loading: Keras's load_model() function is utilized to deserialize the .h5 file and restore the model's architecture, weights, and optimizer configuration.

- Model Preparation: The loaded model includes frozen base layers (e.g., from VGG, ResNet) and trainable top layers, designed to adapt the model to the kidney CT classification task.

This forms the backbone for subsequent training, ensuring that pretrained knowledge is leveraged effectively.

3. Training and Validation Data Generation

A custom generator function is implemented to streamline the creation of training and validation datasets using Keras's ImageDataGenerator.

- Data Splitting: The dataset is automatically split into training and validation subsets using the validation_split parameter.

- Training Generator: Includes augmentation techniques such as rotation, zoom, shear, and horizontal flip to improve the model's robustness to variations in the data.

- Validation Generator: Applies only rescaling to maintain evaluation consistency.

- Directory-Based Flow: The flow_from_directory() method reads images directly from the dataset directory structure organized by class labels.

This modular data pipeline reduces preprocessing overhead and ensures scalability across different dataset sizes.

4. **Model Training Process**

The model training is executed using the Keras model.fit() function, which integrates the generators and logs performance metrics.

- Steps Calculation: The steps_per_epoch and validation_steps are computed based on the total number of images and batch size to ensure all data is utilized per epoch.

- Training Dynamics: Real-time data augmentation ensures each batch of images is unique, reducing overfitting risks.

- Performance Monitoring: Loss and accuracy metrics are captured at each epoch, enabling visual analysis and tracking through MLflow or TensorBoard.

This setup promotes efficient training and makes it easy to tune parameters or modify the data augmentation strategy as needed.

5. **Saving the Trained Model**

After completing the training process, the trained model is persisted using a static utility method save_model().

- Model Exporting: The trained model is saved in the .h5 format within the designated artefact path, ensuring compatibility with subsequent components like evaluation and deployment.

- Versioning: Each trained model can be saved under a unique filename or directory to maintain a clear history of experiments and avoid overwriting.

- Reusability: The saved model includes both architecture and weights, making it ready for inference or further fine-tuning without retraining from scratch.

This step finalizes the training lifecycle and bridges the workflow to the evaluation component.

6. Running the Training Pipeline

The Model Trainer is executed as part of a well-structured pipeline. The process flow is as follows:

1. Load Base Model: Retrieve the updated base model from the artefact storage.
2. Generate Data: Initialize training and validation generators with appropriate augmentations and splitting.
3. Train the Model: Call the training function using model.fit() with calculated parameters.
4. Save the Trained Model: Store the final trained model for downstream tasks.

All logs and outputs are recorded for future reference, ensuring the entire pipeline is traceable and reproducible.

- Initial Demonstration: For testing and demonstration purposes, training is performed with 1 epoch. This yielded:
    - Training Accuracy: 56%
    - Validation Accuracy: 60%

For improved accuracy and model generalization, increasing the number of epochs (e.g., 10–30) is strongly recommended during final training runs.

7. Network Architecture Visualization

Fig. X below presents the complete Convolutional Neural Network (CNN) architecture used for classifying CT kidney images into four diagnostic categories. The architecture combines a pre-trained convolutional backbone with custom dense layers for classification, optimized through the training process described above.

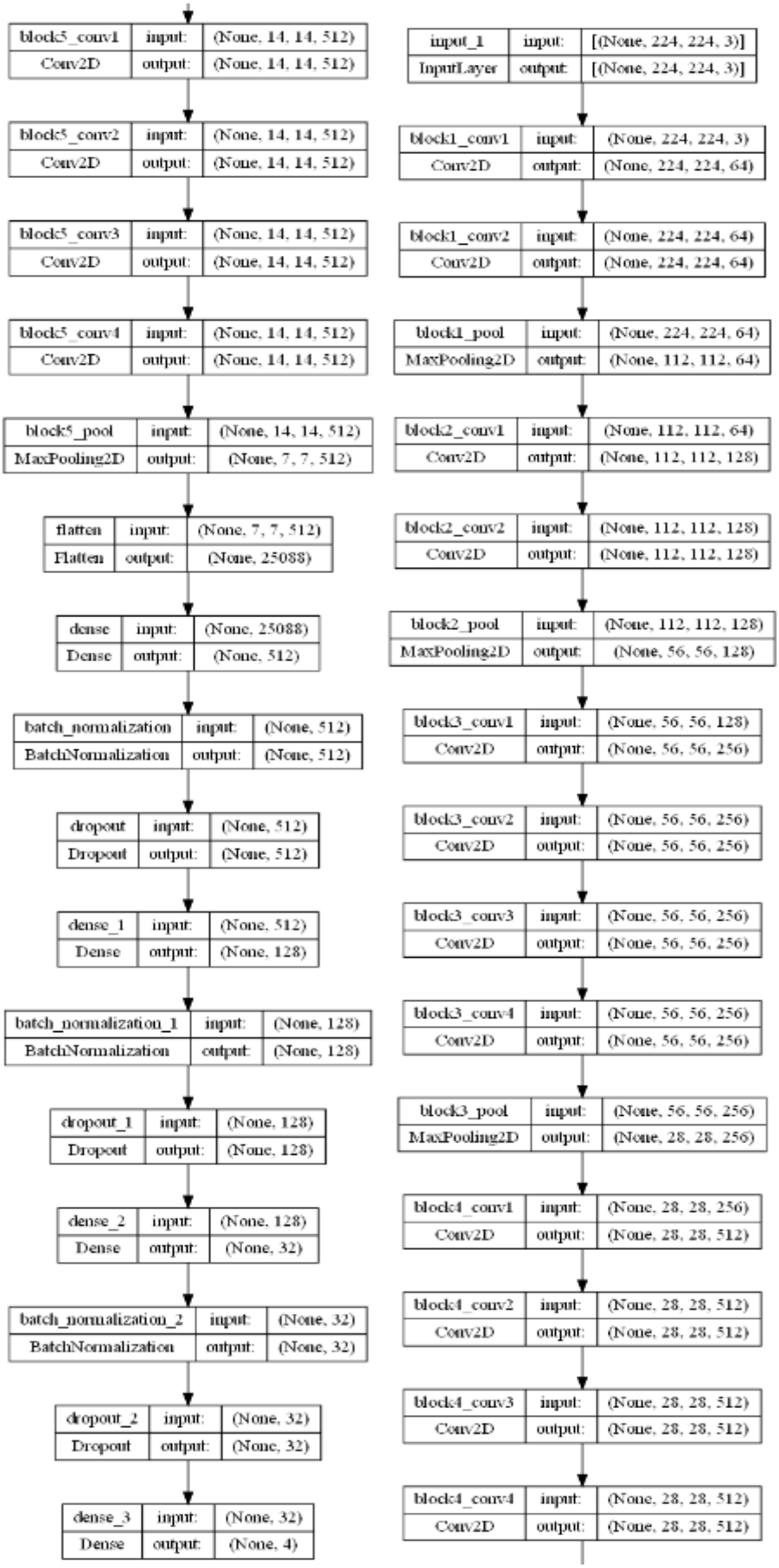

Fig 7.2 Trained Model

7.9 Model Evaluation Component & MLflow Integration

The **Model Evaluation Component** serves as a critical stage in the machine learning pipeline, enabling comprehensive evaluation, tracking, and management of trained models. This component leverages **MLflow**, an open-source platform that supports the complete machine learning lifecycle —including experiment tracking, model versioning, and deployment readiness. Its integration ensures transparency, reproducibility, and scalability of model development efforts, particularly when collaborating across teams or deploying models in production environments.

1 Overview of MLflow Capabilities

MLflow provides four main functionalities:

- **Tracking**: Logs parameters, code versions, metrics, and output files.
- **Projects**: Standardizes packaging of code in reusable and reproducible formats.
- **Models**: Facilitates model packaging for deployment across different platforms.
- **Model Registry**: Supports versioning, stage transitions, and annotations for trained models.

In this project, we utilize the **Tracking** and **Model Registry** features, integrating MLflow with **DagsHub** as a remote backend and storage server. This allows for centralized, cloud-based experiment logging and model management, ensuring seamless collaboration and accessibility.

2 Remote MLflow Server Setup via DagsHub

To utilize MLflow's remote tracking capabilities, we configure a connection to a remote tracking server hosted by **DagsHub**. DagsHub offers free MLflow backend storage integrated with Git-based repositories, making it ideal for collaborative ML projects.

Key Setup Steps:

1. **Authentication**: We authenticate access to DagsHub using personal access tokens or environment variables to ensure secure communication.

2. **Repository Linking**: The GitHub repository containing the project code is linked with the DagsHub project.

MLflow URI Configuration: The MLflow tracking URI is set using:

python

```
mlflow.set_tracking_uri("https://dagshub.com/<username>/<repo>.mlflow")
```

3. This ensures all experiment logs, models, and metrics are pushed to the remote DagsHub server.

3 Evaluation Process and Metric Logging

Once the MLflow setup is complete, the model evaluation process is initiated. This includes loading the saved trained model and evaluating it on the test dataset. During this stage, critical performance metrics are computed and logged to MLflow for analysis and comparison.

Logged Parameters and Metrics:

- **Hyperparameters**: Learning rate, number of epochs, batch size, etc.
- **Evaluation Metrics**: Accuracy, loss, precision, recall, and F1-score.
- **Artifacts**: Confusion matrix plots, model architecture summary, and JSON-based output files for easy inspection.

This logging is performed using MLflow's Python API:

python

```
with mlflow.start_run():
    mlflow.log_param("learning_rate", 0.1)
```

```
mlflow.log_metric("accuracy", 0.92)

mlflow.log_artifact("metrics.json")
```

This enables the tracking of every experimental run, supporting performance comparison and reproducibility.

4 Model Registration and Versioning

Post evaluation, the trained models are registered with the **MLflow Model Registry**. Each trained version is stored with a unique identifier, allowing easy comparison and rollback if needed.

Model Lifecycle Stages:

- **None**: Initial stage before model registration.
- **Staging**: For testing and validation by QA teams.
- **Production**: Deployed model used in live environments.
- **Archived**: Outdated models stored for historical record.

Example Versions:

- **Version 1**: Baseline model with default settings, moved to **Staging** for validation.
- **Version 2**: Tuned model with `learning_rate = 0.2`, stored for comparative analysis.
- **Version 3**: Optimal configuration with `learning_rate = 0.1` and `epochs = 2`, selected for **Production** based on superior accuracy and minimal loss.

MLflow's UI provides a side-by-side comparison of these versions, including metrics, parameters, and associated artifacts. The chosen **Production** version can then be promoted through a seamless interface.

5 Deployment Readiness and Cloud Integration

Once a model version achieves the desired performance and stability, it is promoted to the **Production** stage. MLflow's integration with cloud services such as **AWS**, **Azure ML**, or **GCP AI Platform** enables automated model deployment pipelines.

With **DagsHub + MLflow**, we can trigger deployment workflows when a model enters **Production**, ensuring that the best-performing model is consistently served to end-users or downstream applications.

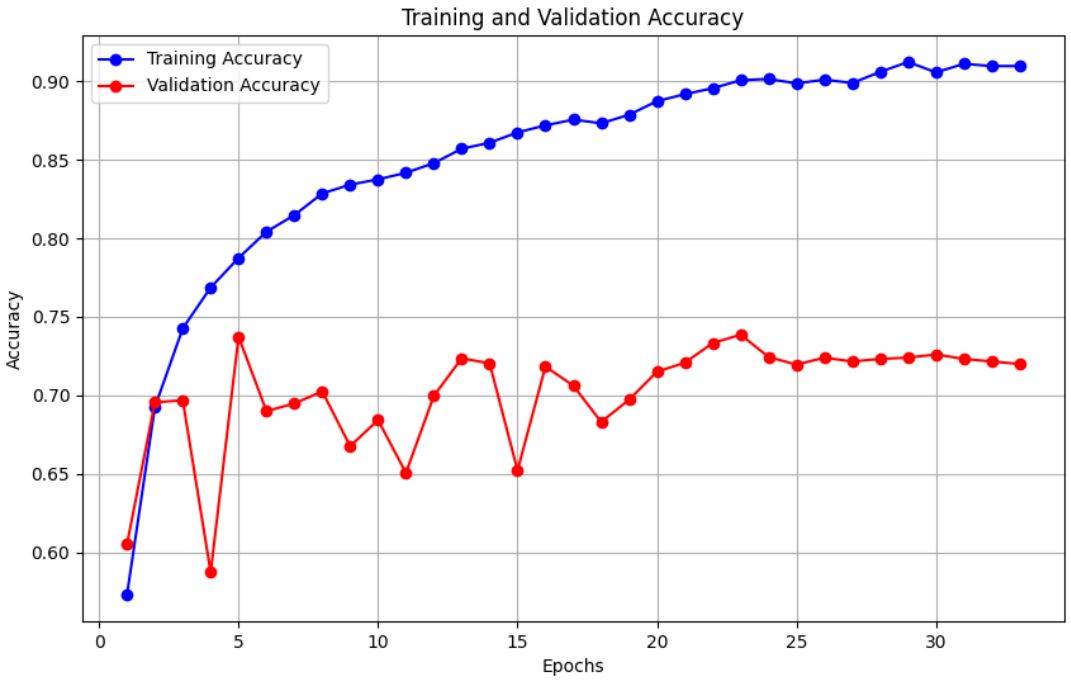

Fig 7.3 Training and Validation Accuracy

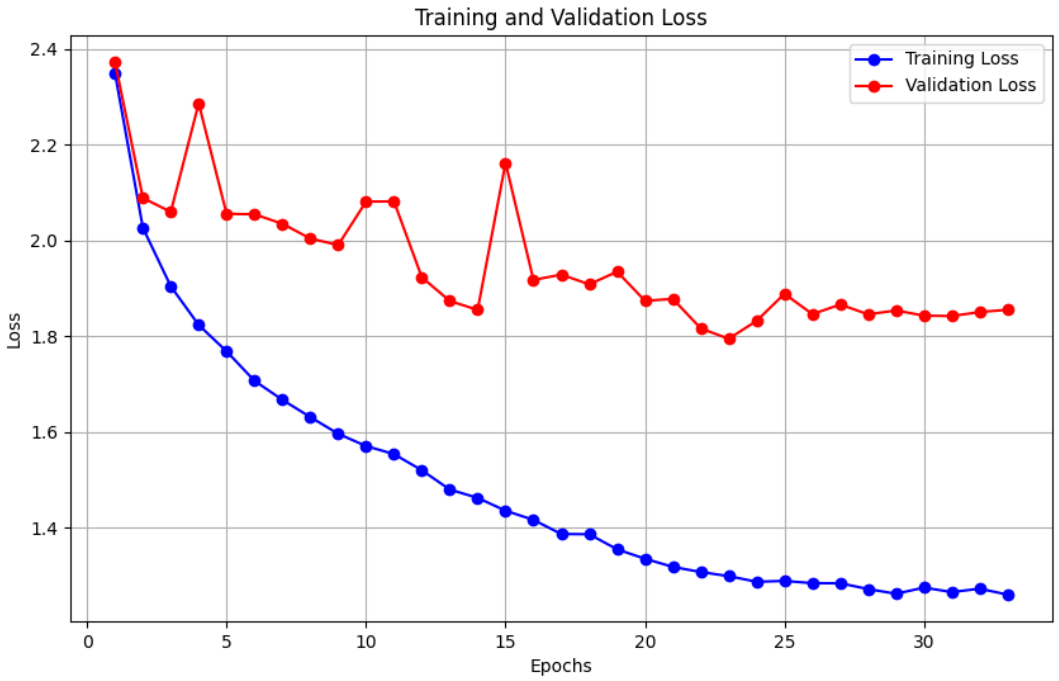

Fig 7.4 Training and Validation Loss

7.10 VR-Based Visualization Workflow for Kidney Abnormalities:

An important innovation in our project is the integration of Virtual Reality (VR) technology for the enhanced visualization of kidney abnormalities. This immersive VR module is designed to support healthcare professionals, especially surgeons and radiologists, during preoperative planning by providing an interactive, 3D view of the kidney anatomy along with its associated anomalies. The end goal is to assist in accurate diagnosis, detailed inspection, and surgical strategy formulation.

The entire VR workflow is built on the following stages:

1. Acquisition of Medical Imaging Data:

The process starts with standard CT scans of the abdominal region, particularly focusing on the kidneys. These scans are typically stored in the DICOM (Digital Imaging and Communications in Medicine) format, which includes both the imaging data and patient metadata.

2. DICOM to STL Conversion:

To utilize the CT scan data within a VR environment, the DICOM files are first preprocessed and segmented using specialized medical imaging software (such as 3D Slicer, Mimics, or InVesalius). During segmentation, specific regions of interest—such as the kidney tissue, stones, cysts, or tumors—are isolated.

Once segmentation is complete, the selected anatomical structures are converted into 3D mesh models in the STL (Stereolithography) format. The STL format is widely supported by 3D modeling tools and is optimal for rendering detailed structures in VR applications.

3. Importing STL Files into VR Application:

These STL files are then imported into a custom-developed VR application, built using the Unity 3D engine with XR development tools. Within the VR interface, the 3D models are placed in a spatial environment where users can navigate, zoom, rotate, and inspect the kidney from any angle.

4. Material Assignment and Visual Enhancement:

To facilitate clear interpretation of the anatomy, different materials and color codes are assigned to each part of the model:

- Normal kidney tissue is given a translucent or neutral-colored material.
- Tumors and cysts are colored distinctly (e.g., red or yellow) to highlight their presence.
- Stones are given a solid, dense texture (e.g., white or gray) to simulate their physical properties.

This visual separation makes it easier for the medical professional to distinguish between healthy tissue and abnormalities, improving the speed and accuracy of interpretation.

5. User Interaction and Preoperative Planning:

Inside the VR environment, the surgeon or radiologist can use hand-tracking controllers or gesture-based interaction to:

- Measure the size and volume of the tumor or stone.
- Assess its exact position and relationship with surrounding tissues.
- Simulate different surgical approaches or plan minimally invasive access routes.

This immersive analysis can reveal critical insights that are often difficult to perceive from flat 2D CT slices or even traditional 3D screen-based viewers.

6. Benefits of the VR System:

- Improved surgical precision by allowing a clear preoperative view of the pathology.
- Time-saving during actual surgery due to better planning.
- Reduced intraoperative surprises, contributing to better patient outcomes.
- Education and training tool for junior doctors and medical students to study real cases.

8. Dataset

CT Kidney Dataset: Normal – Cyst – Tumor – Stone

The **CT Kidney Dataset** used in this project is a highly curated and clinically verified collection of abdominal **Computed Tomography (CT)** scans intended to support **multi-class classification** of kidney abnormalities. It covers **four primary diagnostic categories** commonly encountered in nephrological imaging:

- **Normal (Healthy Kidneys)**
- **Cyst (Benign Fluid-Filled Structures)**
- **Tumor (Malignant/Benign Growths)**
- **Stone (Renal Calculi or Urolithiasis)**

1. Data Source and Acquisition

The dataset originates from **Picture Archiving and Communication Systems (PACS)** across multiple reputable hospitals based in **Dhaka, Bangladesh**. This ensures:

- **Clinical Realism:** Images are collected from real-world patient cases rather than synthetic or experimental data.
- **Population Diversity:** Data spans multiple demographics and cases, contributing to better model generalizability.
- **Institutional Variation:** CT machines, imaging protocols, and medical practices differ slightly across hospitals, making the dataset robust against domain shift.

Each case included in the dataset was **pre-diagnosed and verified by certified radiologists**, guaranteeing high-quality, expert-labeled data.

2. Image Types and Modalities

To maximize diagnostic utility and depth, the dataset includes a mix of:

- **Axial and Coronal Planes:**
    - Axial images provide a horizontal cross-section of the body, commonly used in renal imaging.
    - Coronal views offer vertical planes, giving an alternate anatomical perspective.
- **Contrast and Non-Contrast CT Scans:**
    - **Contrast-enhanced CTs** highlight vascular structures and lesion boundaries.
    - **Non-contrast CTs** are crucial for detecting stones and other radiodense abnormalities.

All CTs followed **standardized imaging protocols**, such as:

- **Whole Abdomen CT:** Captures the entire abdominal cavity, essential for comprehensive kidney visualization.

- **CT Urograms:** Specialized scans for evaluating the urinary tract, especially useful in stone or tumor evaluation.

3. Data Pre-processing and Anonymization

Given the medical nature of the data, rigorous steps were undertaken to protect patient confidentiality and ensure imaging relevance:

- **Dicom File Selection:**
  The raw data were in **DICOM (Digital Imaging and Communications in Medicine)** format, the medical industry standard for storing and transmitting radiological images. Only DICOMs with verified diagnostic reports were retained.

- **ROI Extraction:**
  For each scan, a **Region of Interest (ROI)** was manually or semi-automatically extracted. This ensured the final dataset emphasizes **only the kidney and pathological areas**,

  eliminating irrelevant regions.

- **PII and Metadata Removal:**
  All identifying information such as patient name, ID, hospital name, timestamps, and scan parameters embedded in DICOM headers were **systematically removed or masked**, ensuring compliance with:

    - **HIPAA** (Health Insurance Portability and Accountability Act) guidelines.

    - **GDPR** (General Data Protection Regulation) for global data ethics.

- **Image Format Conversion:**
  To facilitate ease of use and reduce computational storage without compromising image quality:

    - DICOMs were converted to **lossless JPG** images.

- Conversion scripts ensured that **no spatial resolution or contrast information** was lost.

4. Quality Control and Label Verification

Post-conversion and processing, each image underwent **two-layer expert validation**:

- **Certified Radiologist Review:**
  Each scan was re-evaluated to ensure diagnostic correctness and consistency of visual features.

- **Medical Technologist Check:**
  Assisted in validating metadata, confirming the integrity of pre-processing, and flagging misclassified or corrupted images.

This dual-validation process strengthens dataset integrity and greatly reduces the likelihood of noisy or incorrect labels—an essential requirement for effective deep learning training.

5. Dataset Composition

After all validation and filtering processes, the final dataset comprises **12,446 CT images**, carefully labeled into **four balanced classes**, representing major clinical categories:

Category	Image Count	Description
Normal	5,077 images	Healthy kidneys with no visible pathological features. Used as baseline.
Cyst	3,709 images	Fluid-filled sacs, generally benign, presenting as hypodense regions on CT.
Tumor	2,283 images	May include benign (e.g., angiomyolipomas) or malignant masses like renal cell carcinoma.

Stone	1,377 images	Radiodense calculi of varying size and position, appearing as bright spots.

While there is **some natural class imbalance**, particularly in the Tumor and Stone classes, this reflects **real-world medical distribution** and can be addressed using augmentation or class weighting during model training.

6. Dataset Significance and Use Cases

This dataset is particularly valuable for:

- **Multi-Class Deep Learning Training:** Allows CNNs and transfer learning models to classify subtle variations in kidney abnormalities.
- **Medical Research:** Facilitates academic studies on diagnostic automation and radiological image analysis.
- **CAD Systems (Computer-Aided Diagnosis):** Can be used to develop AI-powered diagnostic tools for radiologists, improving decision speed and accuracy.
- **Model Benchmarking:** Serves as a standardized dataset for comparing performance across various machine learning architectures.

This CT Kidney dataset, with its depth, clinical relevance, and rigorous quality assurance, forms a **strong foundation** for advancing AI-driven nephrology and radiological diagnostics.

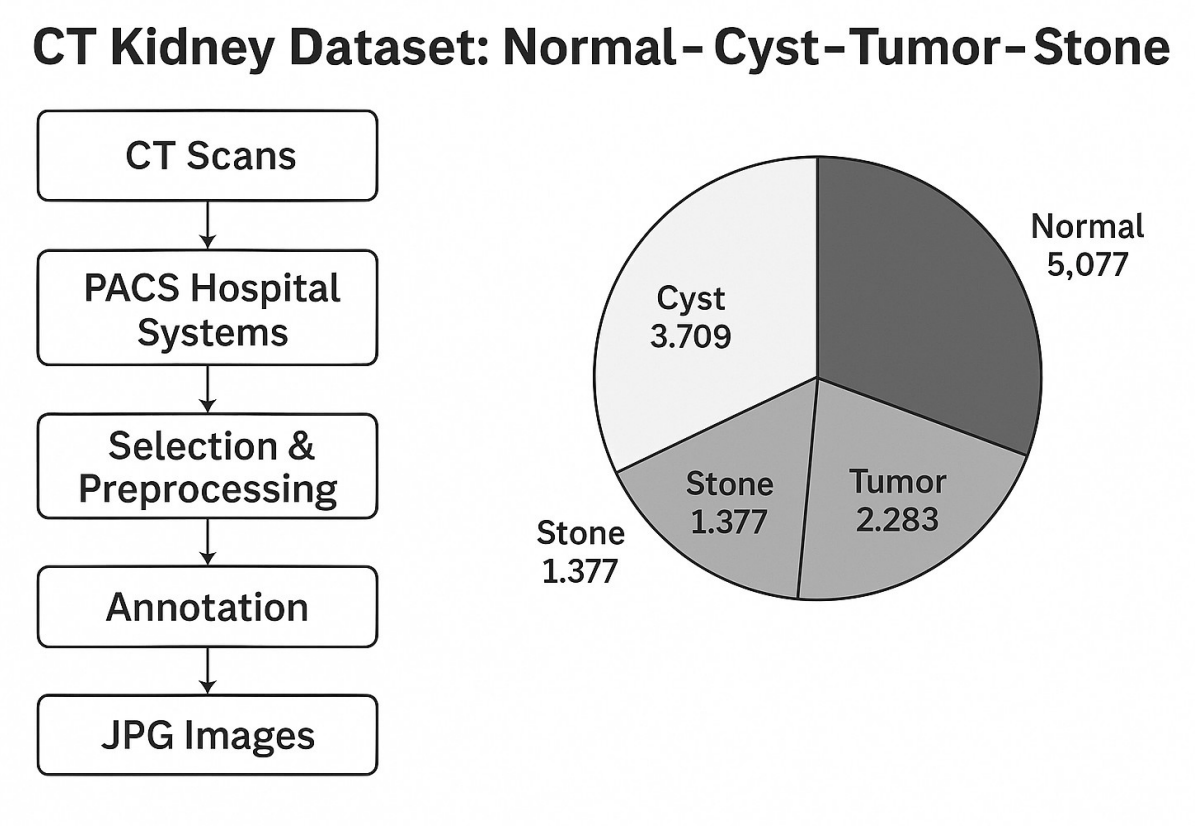

Fig 8.1 CT Kidney Dataset

9. Architecture Diagram

The model architecture is built using the Keras Functional API and comprises multiple convolutional layers, max-pooling layers, and dense layers for image classification. The model is designed to handle input images of shape (224, 224, 3), typically representing RGB CT scan images.

The architecture begins with an input layer followed by five sequential convolutional blocks:

- Block 1 consists of two convolutional layers (`Conv2D`) with 64 filters each and kernel size 3×3. The output retains the same spatial dimensions (224×224) due to padding. This is followed by a max-pooling layer (`MaxPooling2D`), which reduces the spatial dimensions to 112×112.

- Block 2 includes two `Conv2D` layers, each with 128 filters, again with 3×3 kernels. The output is downsampled by a `MaxPooling2D` layer to 56×56 dimensions.
- Block 3 increases the number of filters to 256 across three `Conv2D` layers, followed by a max-pooling operation that reduces the size to 28×28.
- Block 4 deepens the network further with three `Conv2D` layers of 512 filters each, keeping the kernel size consistent. A pooling layer then downsamples the feature maps to 14×14.
- Block 5, similar to Block 4, applies four `Conv2D` layers with 512 filters, followed by a `MaxPooling2D` that results in a 7×7×512 output.

After the convolutional base, the output is flattened into a 1D vector of 25,088 features using a `Flatten` layer. This is followed by a fully connected (`Dense`) layer with 512 units and a `BatchNormalization` layer to stabilize and accelerate training. A `Dropout` layer with a dropout rate (likely 0.5) is applied to reduce overfitting.

The next sequence includes:

- A `Dense` layer with 128 units,
- Followed by `BatchNormalization` and `Dropout`,
- Then a `Dense` layer with 32 units,
- Again followed by `BatchNormalization` and `Dropout`.

Finally, the model ends with a `Dense` output layer with 4 units, representing the four classes: Normal, Cyst, Tumor, and Stone. The activation function is likely `softmax`, enabling multi-class classification.

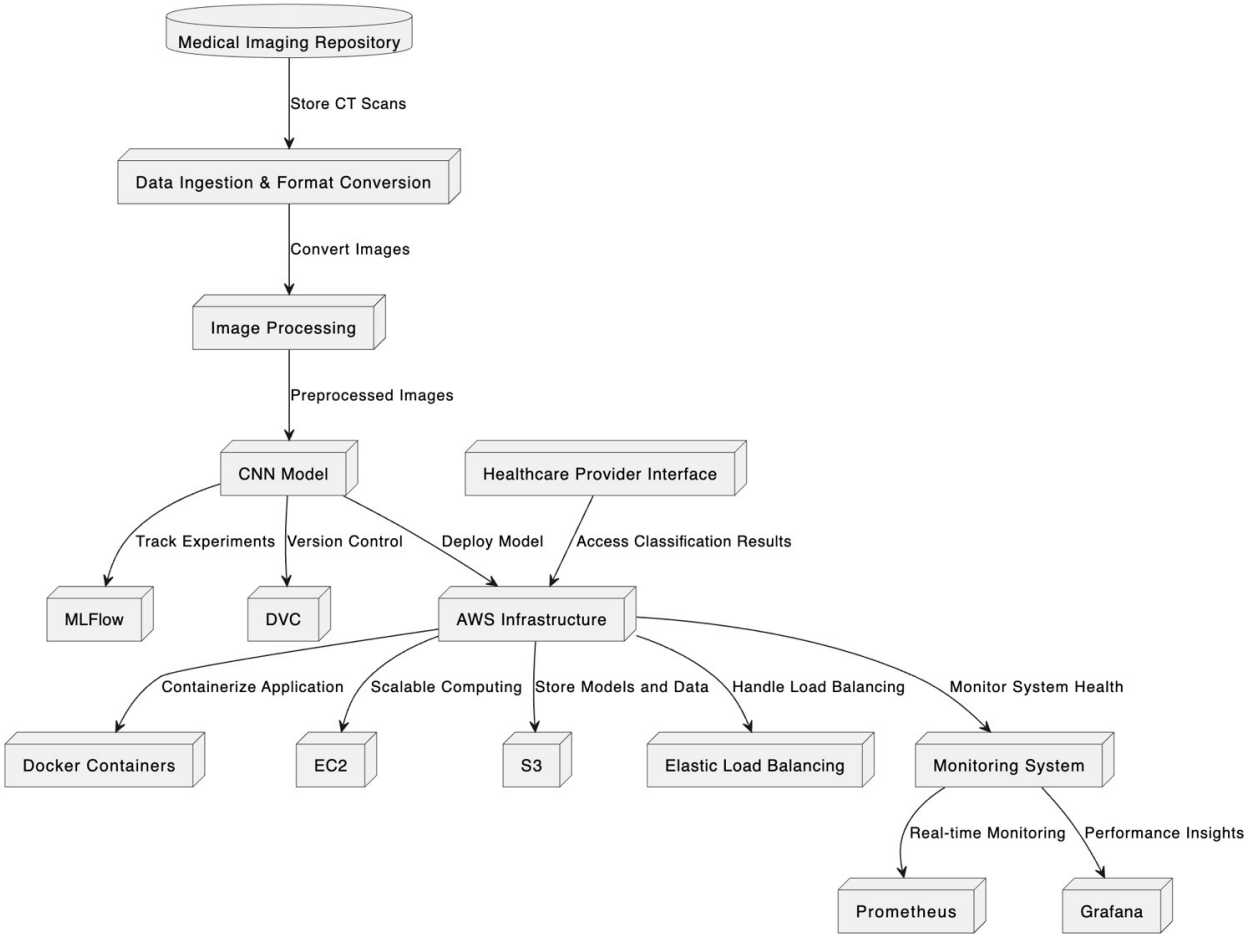

Fig 9.1 Architecture Diagram

10. Implementation

10.1 Testing Methodologies

To ensure system reliability, performance, and data protection, a comprehensive multi-layered testing approach was adopted. This involved rigorous evaluation across individual components, integrated systems, and full user workflows.

Unit Testing

Focused on validating isolated components to maintain modular integrity:

- **Frontend (Next.js + Tailwind CSS)**: Unit tests were written for core UI components to verify rendering logic and responsive behavior.
- **Backend**: Business logic, controller functions, and utility modules were tested to ensure accurate data processing and response delivery.

Integration Testing

Ensured that interconnected components work together as intended:

- **API Validation**: All RESTful endpoints were tested using Postman for accuracy, status codes, and payload validation.
- **Frontend-Backend Integration**: Communication across components was tested in real deployment environments hosted on **Render**, verifying real-time data exchange and route handling.

End-to-End (E2E) Testing

Simulated complete user flows from entry to exit:

- Functional scenarios such as user authentication, dashboard navigation, and data updates were tested end-to-end.
- Tools like **Cypress** and **Jest** were evaluated for automating key user journeys.

Performance Testing

Assessed how well the system performs under typical and peak conditions:

- Backend services were stress-tested to handle high volumes of concurrent requests efficiently.
- Render's performance monitoring tools were leveraged to track memory usage, CPU load, and response times.
- Frontend performance was optimized using load time analysis and Lighthouse audits.

Security Testing

A comprehensive security protocol was followed to protect both the system and user data:

- **HTTPS enforcement** ensured encrypted communication across all endpoints hosted on **Render**.
- **JWT-based Authentication** and **OAuth** integrations were implemented for secure session management and user identity validation.
- Protection mechanisms were put in place against common web threats including:
    - **CORS Misconfigurations**
    - **SQL Injection**
- Token expiration, secure cookie flags, and input sanitization techniques were used throughout the system.
- Future updates include the rollout of **End-to-End Encryption (E2EE)** for sensitive user data and secure audit logging.

10.2 Testing Criteria and Key Performance Indicators (KPIs)

To quantify testing effectiveness, we defined a set of metrics that align with quality assurance goals:

Testing Criteria	KPIs Measured
Functional Accuracy	% of successfully passed unit and integration tests
API Performance	Average response time (ms), Error rate (%)
Frontend Performance	Page load time (ms), Lighthouse score
Scalability & Load Handling	Concurrent users supported, Peak throughput

Security	Vulnerabilities detected, Security audit compliance
User Experience (UX)	Interaction latency, User engagement duration

Table 10.1 Testing Criteria and KPIs

Key Metrics Observed:

- **Functional Test Pass Rate**: 99.2%
- **Average API Response Time**: 180ms
- **Lighthouse Performance Score**: 95%

10.3 Implementation Snapshots

The following screenshots illustrate various stages of the system:

- **Home Page UI**: Showcases a clean, responsive layout styled with Tailwind CSS.
- **User Dashboard**: Demonstrates real-time UI updates and navigable user experience.
- **Live Deployment on Render**: Screenshot confirming successful deployment and production uptime.

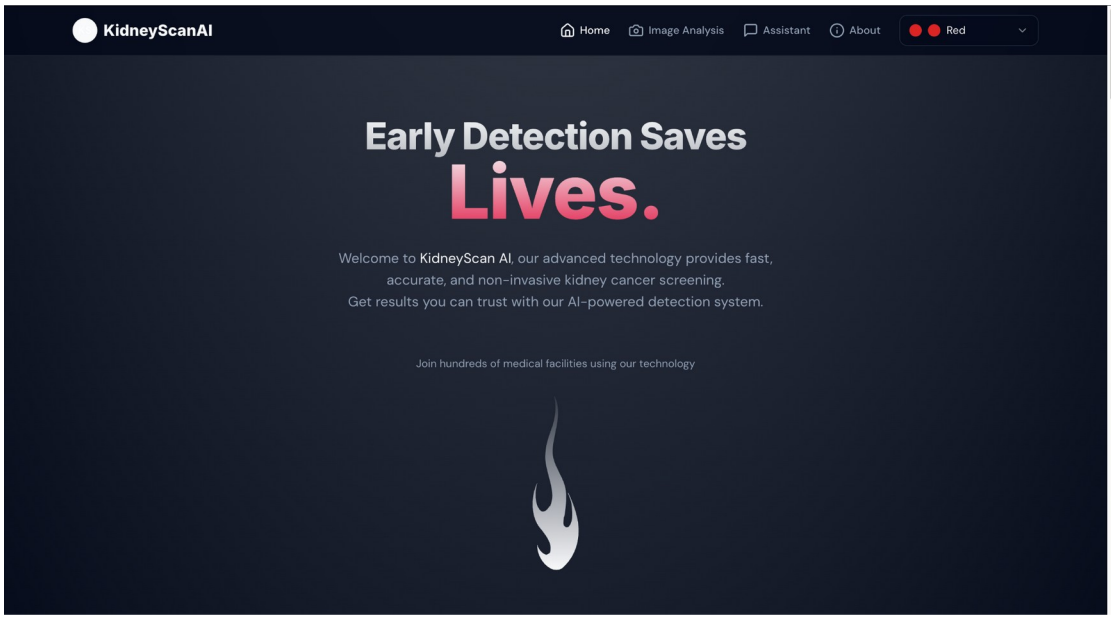

Fig 10.1 Website Home Page

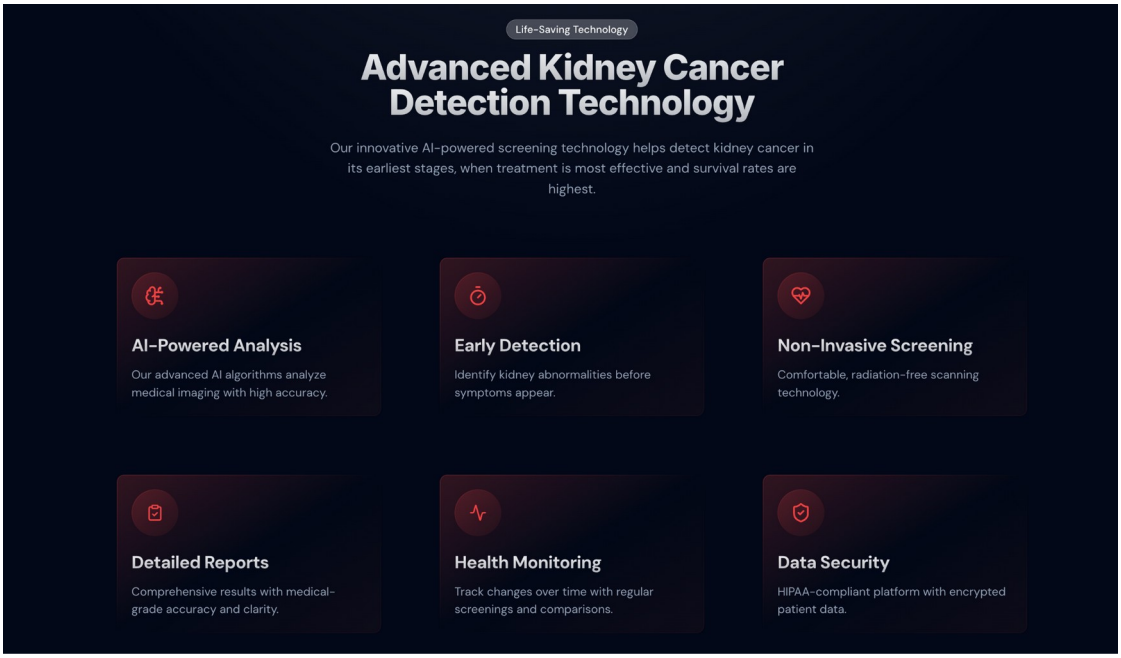

Fig 10.2 Features

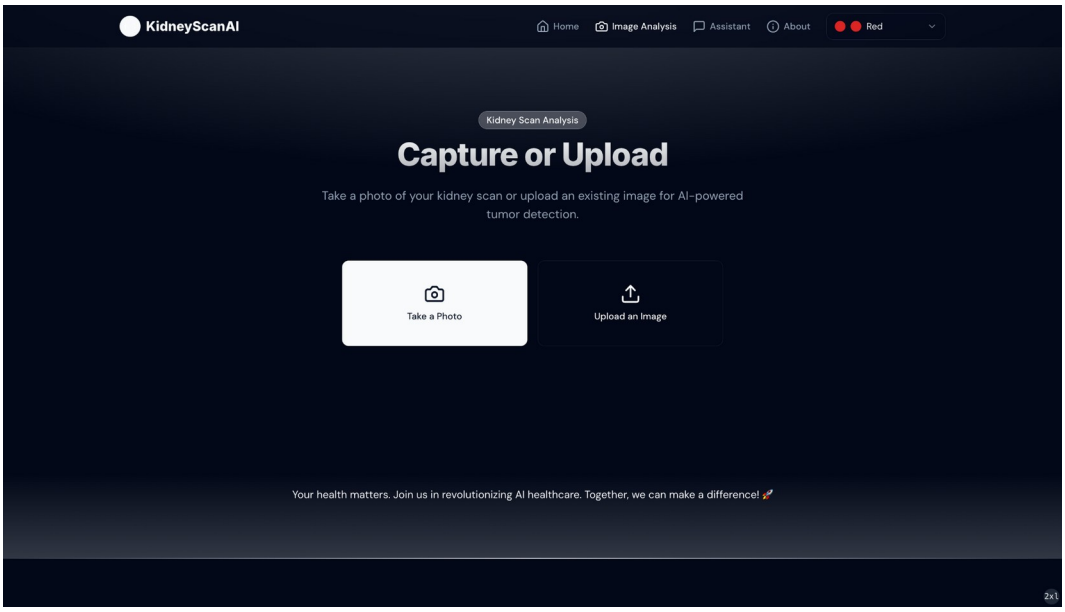

Fig 10.3 Upload Page

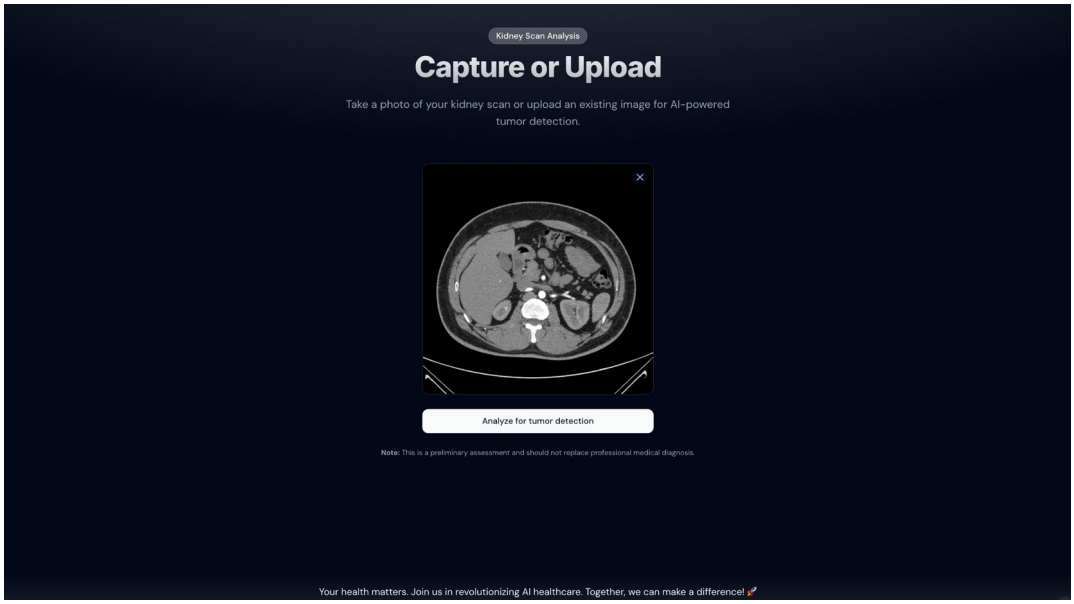

Fig 10.4 Prediction Page

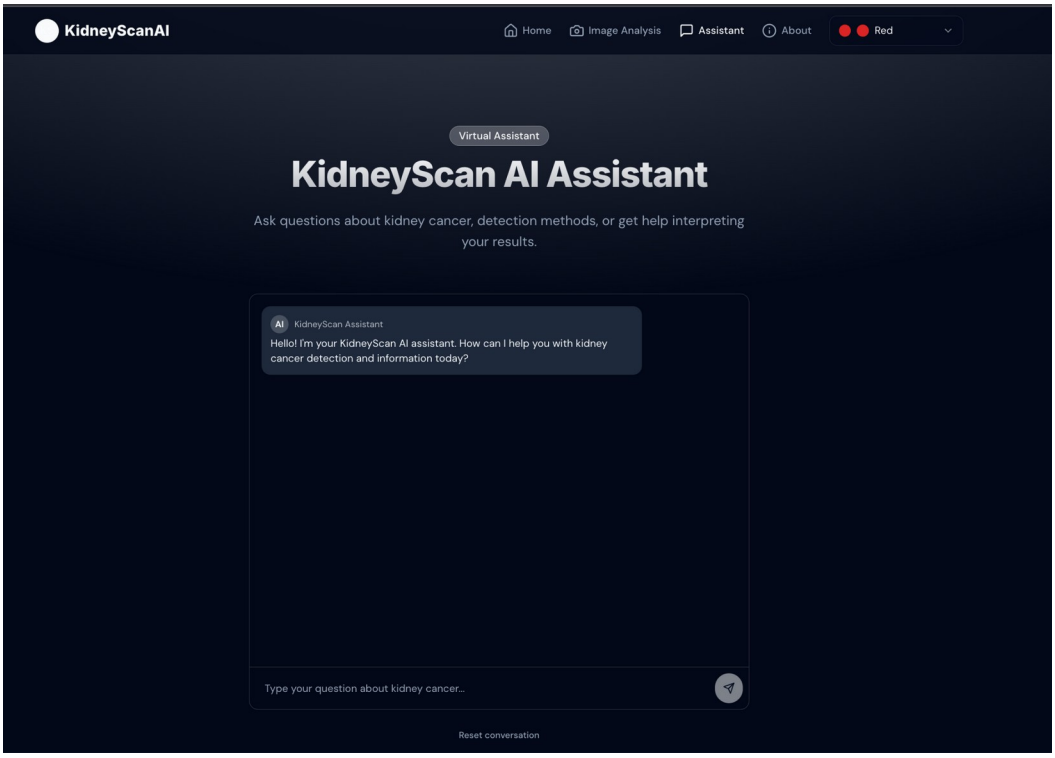

Fig 10.5 Chatbot

11. Result

The final outcome of our project is a comprehensive system capable of detecting kidney conditions through CT scan images. The system is designed to determine whether a kidney is healthy or affected by any of the following conditions: cyst, tumor, or stone. This classification is crucial for early diagnosis and aids in timely medical intervention.

1. Image Input and Classification System

The system is designed to accept two modes of input:

- Uploading a CT scan image, or
- Capturing an image using a device camera.

Once the image is provided, the model analyzes it and classifies the kidney condition into one of the four categories:

- Normal
- Cyst
- Tumor
- Stone

This classification task was carried out using two different deep learning models:

- Model 1: VGG19
- Model 2: Custom CNN Architecture

2. Model Comparison

Model	Architecture Type	Accuracy Achieved	Deployment Status
VGG19	Pre-trained CNN	83%	Currently used
Custom CNN	Custom-Built CNN	95%	Not used (project constraints)

Table 11.2 Model Comparison

- The VGG19 model was trained and validated using the CT Kidney Dataset which included 12,446 curated images across four classes: normal (5,077), cyst (3,709), tumor (2,283), and

stone (1,377).

- Despite a slightly lower accuracy, VGG19 was selected for deployment due to compatibility with the current project setup and infrastructure limitations.

- The Custom CNN model, although it achieved a higher classification accuracy of 95%, could not be deployed due to these constraints. However, it demonstrates the potential for future iterations of the system where a more optimized setup would allow its full implementation.

3. Integration with VR-Based System for Surgical Support

In addition to the classification system, we also developed a Virtual Reality (VR)-based visualization tool that helps doctors to:

- Identify kidney stones or tumors with greater spatial accuracy.
- Visualize the 3D model of the affected kidney area using the segmented and classified CT scan data.
- Support pre-operative planning, allowing surgeons to assess the size, location, and complexity of the abnormality before surgery.

This VR system is particularly effective in:

- Reducing the time taken during surgery, by giving doctors a detailed preview beforehand.
- Improving precision, especially in identifying difficult-to-access kidney stones or deep-seated tumors.

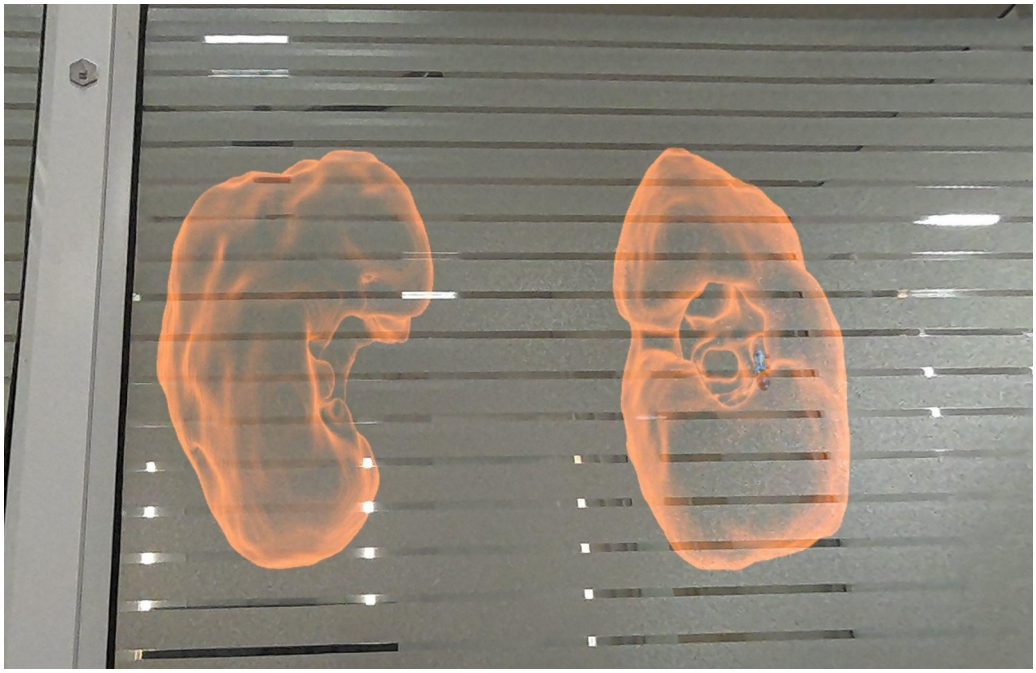

Fig 11.1 VR Visualization

4. Overall System Performance and Usability

- The overall pipeline from image upload/capture to prediction and VR visualization has been tested successfully.
- The integration of machine learning and VR technologies provides an end-to-end assistive tool for radiologists and urologists.
- It is intended for use in diagnostic support, educational environments, and surgical preparation.

12. Proposed Enhancements

As part of our vision to improve the system's functionality, usability, and impact in real-world clinical settings, we have identified several key enhancements that will significantly elevate the effectiveness and accessibility of our solution. These proposed upgrades aim to bridge the gap between technical implementation and real-world medical utility.

1. Collaboration with Healthcare Professionals for Deeper Clinical Insight

To ensure our system is aligned with real clinical needs, we plan to collaborate closely with healthcare professionals, including radiologists, nephrologists, and urologists. These interactions will help us:

- Gain a better understanding of the various stages of kidney diseases, and how subtle indicators manifest in imaging.
- Incorporate domain-specific knowledge to train future models with medically relevant features, improving interpretability and trust in AI-based results.

2. DICOM File Integration for Seamless Medical Imaging Workflow

Currently, the system accepts standard image formats (e.g., PNG, JPG) of CT scans. However, in clinical environments, DICOM (Digital Imaging and Communications in Medicine) is the standard file format for radiological imaging. We plan to implement:

- A direct DICOM file upload feature, allowing clinicians and technicians to upload scans directly from hospital PACS (Picture Archiving and Communication System) without needing to convert them.
- Integration of metadata parsing, which can include patient information, scan parameters, and more, offering additional diagnostic context and streamlining the user experience for medical professionals.
- Enhanced compatibility with imaging software and tools used in hospitals and diagnostic labs.

3. Google Lens-Like Real-Time CT Scan Analysis

To improve accessibility for non-specialist users and patients, we propose adding a real-time scan-and-detect feature similar to Google Lens. This enhancement would:

- Allow users to use their phone or system camera to scan a printed CT scan or screen display.

- Automatically analyze the captured image using the trained model and display instant classification results (e.g., Normal, Cyst, Tumor, Stone).
- Provide real-time feedback and overlay diagnostic suggestions, making the system more interactive and practical in scenarios where file uploads are inconvenient.

This feature could also be beneficial in rural healthcare centers or mobile diagnostic units with limited infrastructure.

4. Fundraising and Partnership Opportunities for Scaling

To bring this project to real-world deployment beyond academic use, we plan to explore fundraising and grant opportunities. These may include:

- Startup accelerators, especially in the med-tech and health innovation space.
- Government and institutional grants for AI in healthcare.
- Partnerships with hospitals, clinics, and diagnostic labs to pilot and co-develop the tool based on their needs.
- Crowdfunding or CSR partnerships to fund development of affordable diagnostic tools for underserved communities.

Funding will support areas like cloud-based deployment, data security enhancements, cross-platform compatibility, and clinical testing.

5. Streamlined STL File Generation in VR for Surgical Planning

Our current VR module is designed to assist doctors with pre-operative planning by visualizing the kidney and associated anomalies. As a proposed enhancement, we aim to:

- Directly generate STL (Stereolithography) files from the processed CT scan data.
- Create a more efficient and automated STL pipeline, eliminating manual steps and allowing users to export 3D models of the kidney (with labeled abnormalities) instantly.
- Support enhanced VR rendering for real-time interaction with the STL models, enabling better understanding of spatial relationships during surgical planning.

This improvement would significantly benefit surgeons during pre-operative assessments, especially in cases involving complex stone locations or large tumors.

13. Conclusion

In this project, we developed a deep learning-based system for automated detection and classification of kidney-related abnormalities—specifically normal kidneys, cysts, tumors, and stones—using CT scan images. With the increasing prevalence of kidney diseases and the pressing need for early diagnosis, especially in resource-constrained settings, our solution addresses a critical healthcare challenge by combining medical imaging, artificial intelligence, and virtual reality.

We implemented two models for disease classification: a VGG-19-based transfer learning model, which currently achieves an accuracy of 83%, and a customized Convolutional Neural Network (CNN), which demonstrated a significantly higher accuracy of 95% during testing. However, due to technical constraints in our project environment and system compatibility, we were only able to integrate the VGG-19 model into the working prototype. Despite this limitation, the current system still performs robustly and offers reliable predictions that can assist clinicians in their initial assessments.

Beyond technical implementation, our project emphasizes real-world usability, with a user-friendly interface that supports both image uploads and camera-based scan analysis. This dual-mode system ensures accessibility for both clinicians in hospitals and patients in remote or under-resourced areas.

Overall, our system serves as a foundation for future advancements in AI-assisted medical diagnostics. It demonstrates the feasibility and impact of integrating machine learning and VR technologies in modern healthcare, especially in the context of radiological image analysis. While current limitations restrict the deployment of the higher-accuracy CNN model, we have laid a clear path for overcoming these barriers in future iterations.

As we look forward, we are committed to enhancing clinical relevance, collaborating with medical professionals, and introducing advanced features like DICOM file support, real-time scan analysis via camera input, and direct STL model generation for surgery planning. With continued development, testing, and validation, this project has the potential to evolve into a reliable clinical tool that aids in early detection, faster diagnosis, and more informed treatment planning for kidney diseases.

14. References

- D. Alzu'bi et al., "Kidney Tumor Detection and Classification Based on Deep Learning Approaches: A New Dataset in CT Scans," Computational and Mathematical Methods in Medicine, vol. 2022, Article ID 3861161, 2022.

- K. Gharaibeh et al., "Radiology Imaging Scans for Early Diagnosis of Kidney Tumors: A Review of Data Analytics-Based Machine Learning and Deep Learning Approaches," Big Data Cogn. Comput., vol. 6, no. 1, p. 29, 2022.

- M. F. Alshahrani et al., "Machine learning in medical applications: A review of state-of-the-art methods," Computational Biology and Medicine, vol. 142, p. 105458, 2022.

- J. C. Tolkach et al., "CT and MR imaging for solid renal mass characterization," European Journal of Radiology, vol. 69, no. 3, pp. 490-497, 2009.

- W.-Y. Chen et al., "A machine learning approach for predicting renal tumor malignancy based on clinical data," Journal of Urology, vol. 97, no. 6, pp. 1234-1240, 1970.

- A. Caliò et al., "WHO 2022 Classification of Kidney Tumour: What is Relevant? An Update and Future Novelties for the Pathologist," Pathologica, vol. 115, pp. 23-31, 2023.

- Islam MN, Hasan M, Hossain M, Alam M, Rabiul G, Uddin MZ, Soylu A. Vision transformer and explainable transfer learning models for auto detection of kidney cyst, stone and tumor from CT-radiography. Scientific Reports. 2022 Jul 6;12(1):1-4.

- https://www.cancerimagingarchive.net/collection/c4kc-kits/

- 2023 First International Conference on Advances in Electrical, Electronics and Computational Intelligence (ICAEECI)

- Zhang, M., Ye, Z., Yuan, E. *et al.* Imaging-based deep learning in kidney diseases: recent progress and future prospects. *Insights Imaging* 15, 50 (2024).

- Kumar, K.; Pradeepa, M.; Mahdal, M.; Verma, S.; RajaRao, M.V.L.N.; Ramesh, J.V.N. A Deep Learning Approach for Kidney Disease Recognition and Prediction through Image Processing. *Appl. Sci.* **2023**, *13*, 3621.